

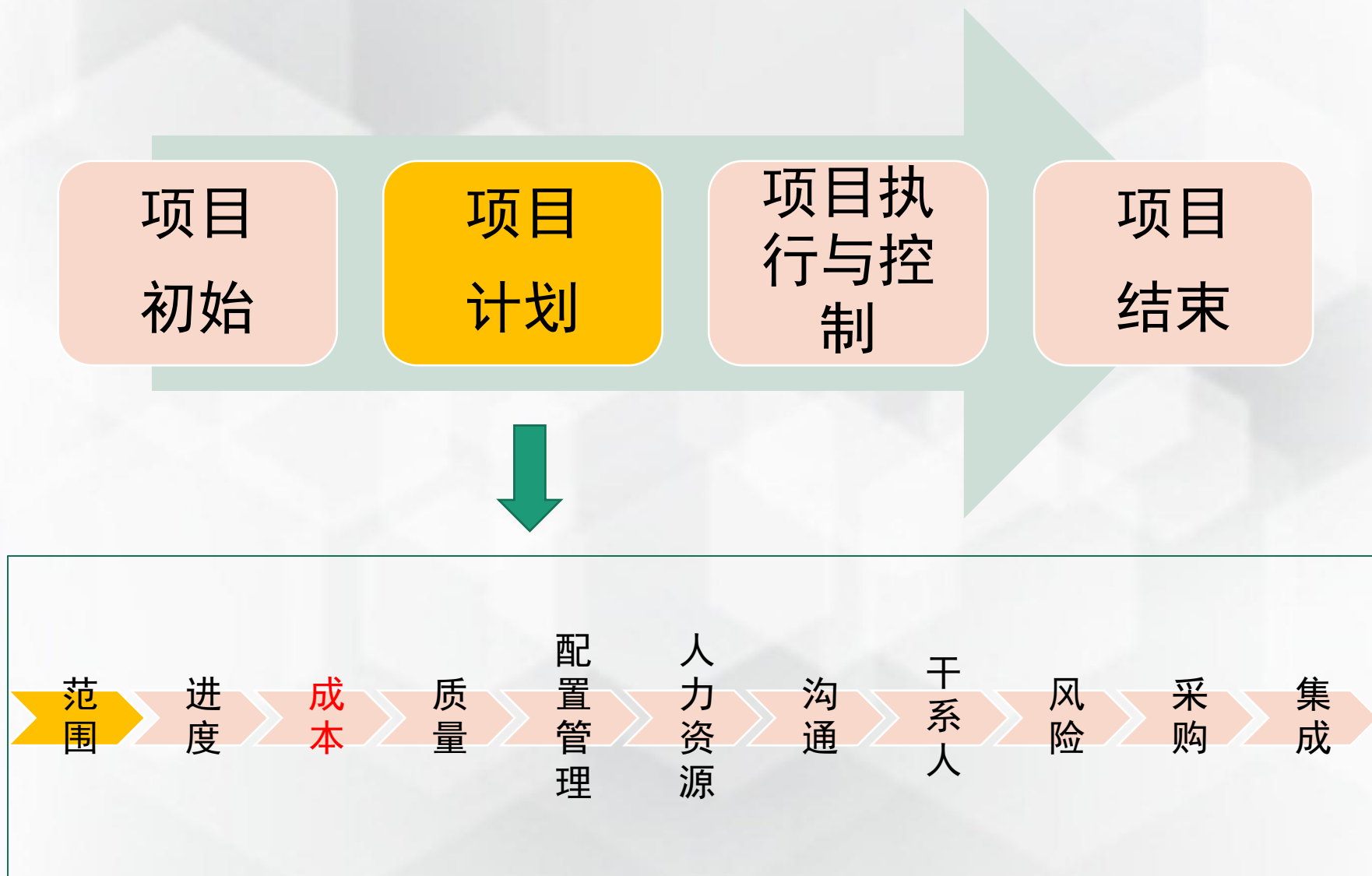


第6章

软件项目成本计划



软件项目管理过程





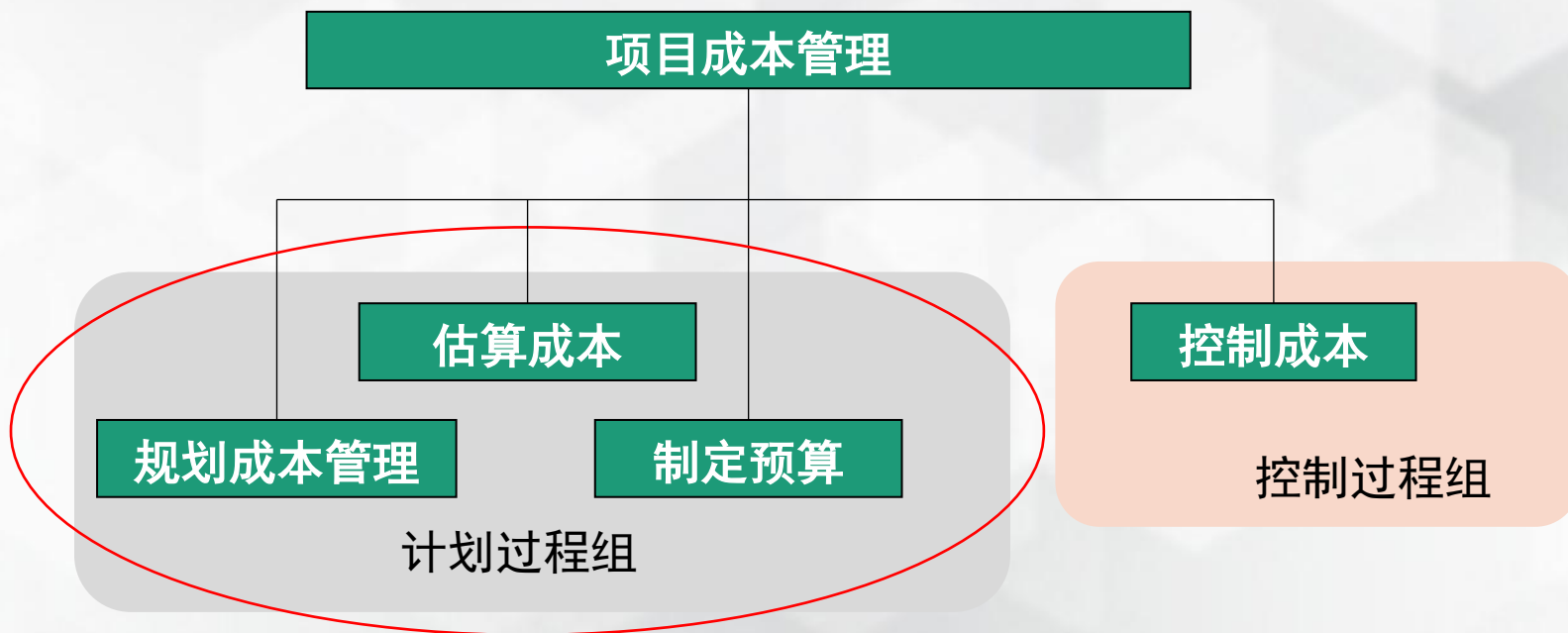
项目成本管理

过程组/知识领域	启动过程组	计划过程组	执行过程组	控制过程组	收尾过程组
项目集成管理	编制项目章程	编制项目管理计划书	指导与管理项目执行	监控项目工作 实施集成变更控制	结束项目或阶段
项目范围管理		规划范围管理 收集需求 定义范围 创建任务分解结构		确认范围 控制范围	
项目时间管理		规划进度管理 定义活动 排列活动顺序 估算活动资源 估算活动持续时间 制定进度计划		控制进度	
项目成本管理		规划成本管理 估算成本 制定预算		控制成本	
项目质量管理		规划质量管理	实施质量保证	控制质量	
项目人力资源管理		规划人力资源管理	组建项目团队 建设项目团队 管理项目团队		
项目沟通管理		规划沟通管理	管理沟通	控制沟通	
项目风险管理		规划风险管理 识别风险 实施定性风险分析 实施定量风险分析 规划风险应对		控制风险	
项目采购管理		规划采购管理	实施采购	控制采购	结束采购
项目干系人管理	识别干系人	规划干系人管理	管理干系人	控制干系人	



什么是项目成本管理?

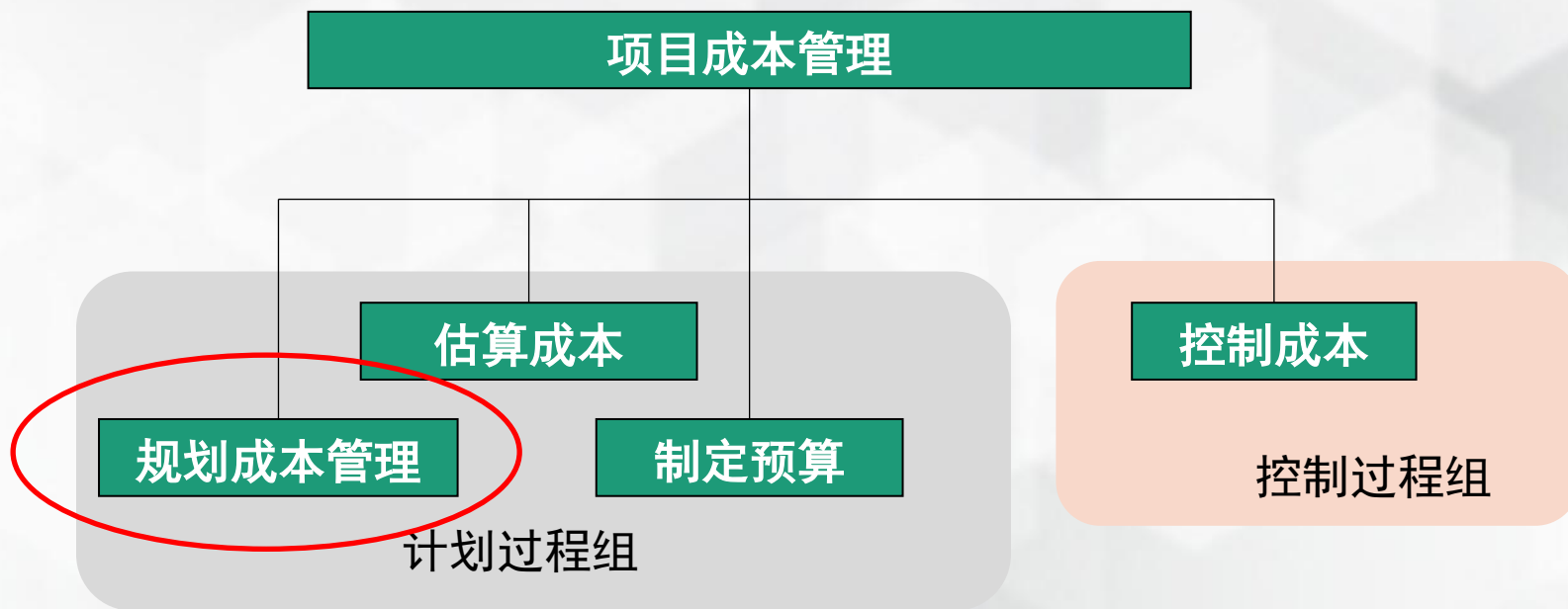
项目成本管理，就是为了在审批过的项目预算内完成项目，所进行的成本计划、估算、预算、监视和控制等一系列管理活动。





什么是项目成本管理?

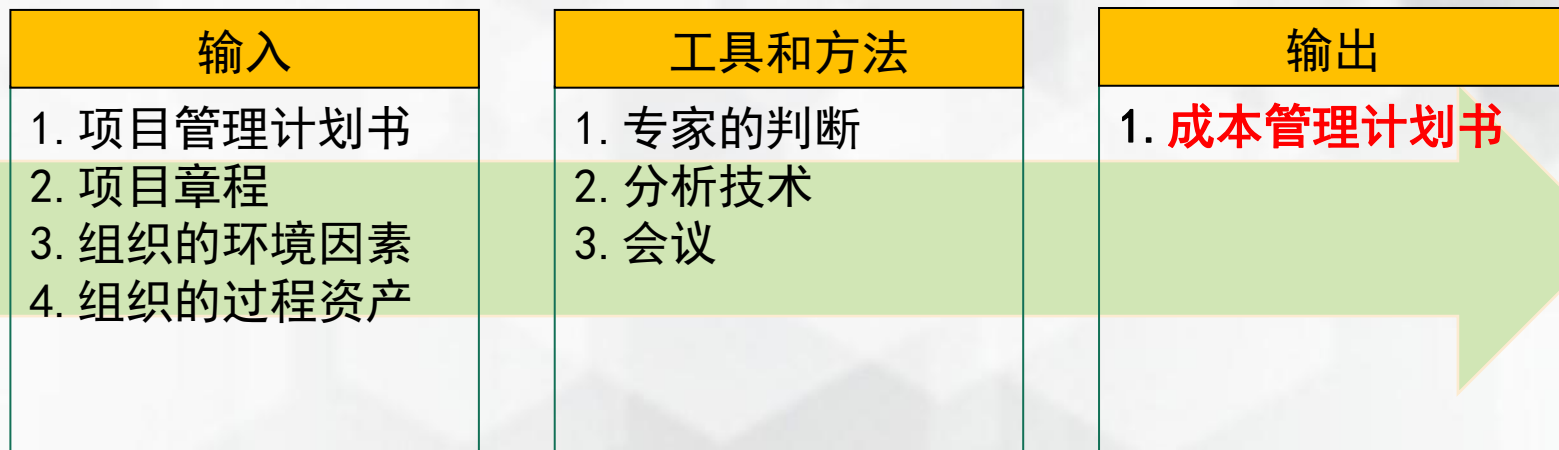
项目成本管理，就是为了在审批过的项目预算内完成项目，所进行的成本计划、估算、预算、监视和控制等一系列管理活动。





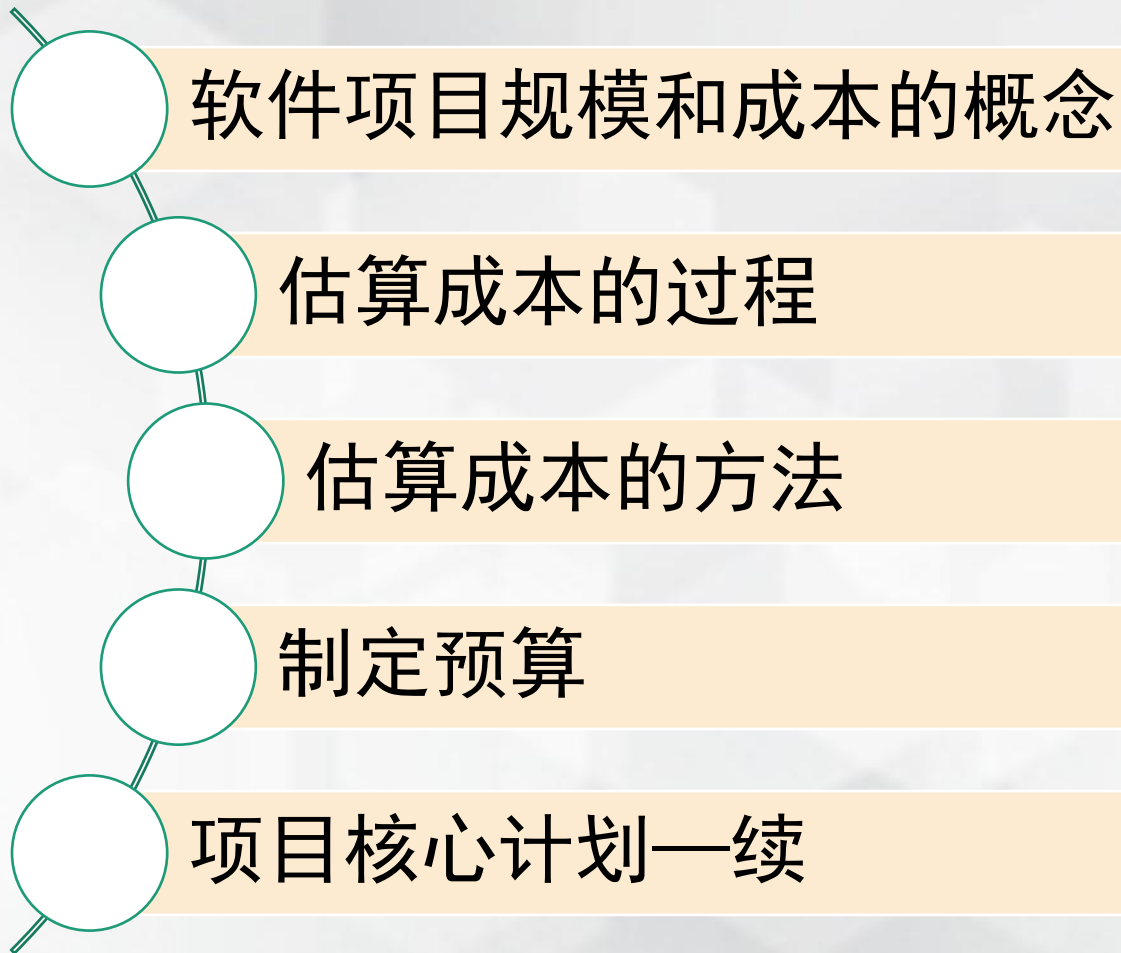
规划成本管理

规划成本管理的过程，是来决定如何对项目成本进行管理为主要目的的活动。



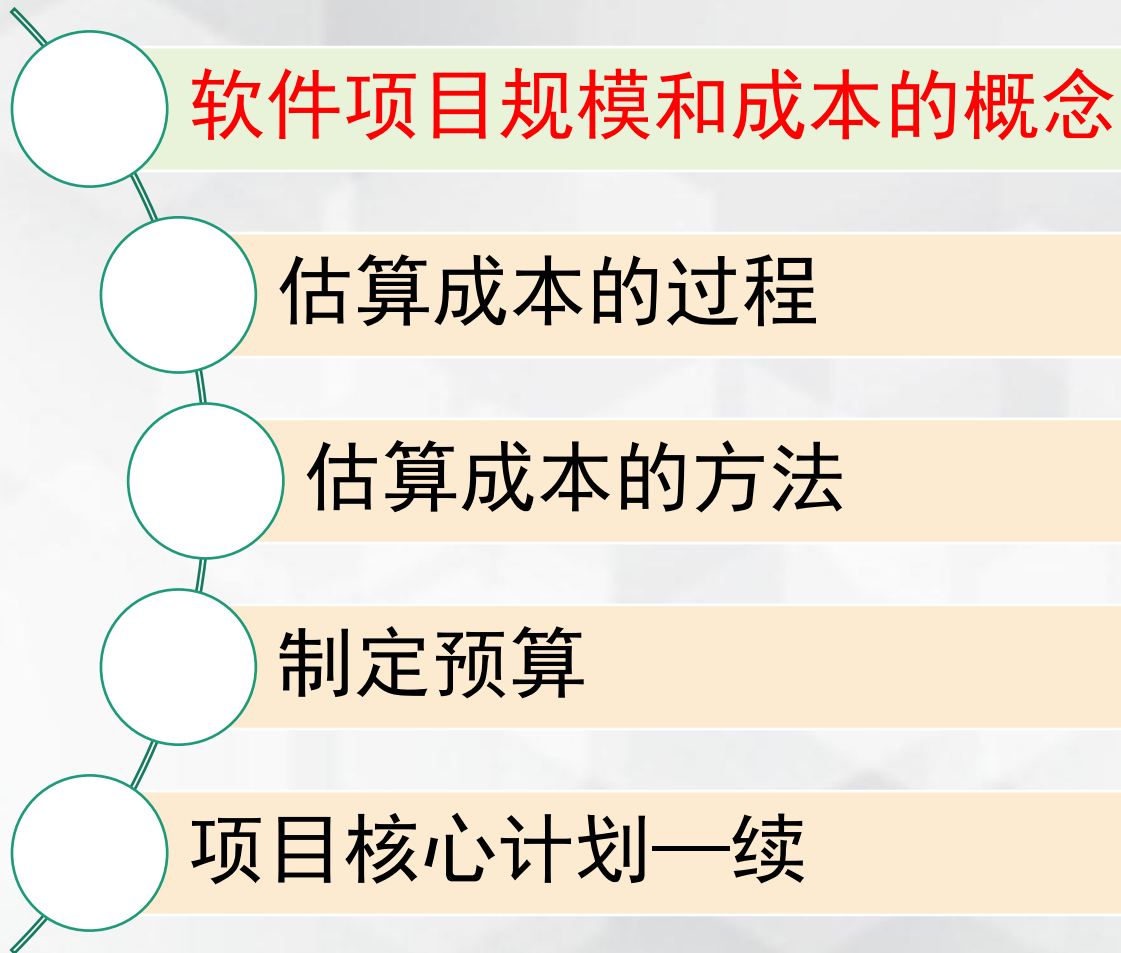


本章内容





本章内容





软件项目规模

- 软件项目规模即**工作量**，是从软件项目范围中抽出的软件功能，然后确定每个软件功能所必须执行的一系列软件开发工作的任务量。
- 工作包括：软件规划、软件管理、需求、设计、编码、测试，以及后期维护等。



软件项目规模的单位

➤ LOC (Lines Of Code)

源代码程序长度的测量

➤ FP (Function Point)

用系统的功能数量来测量

➤ 人月

➤ 人天

➤ 人年



软件项目成本

完成软件规模所相应付出的代价。

- 指待开发的软件项目需要的资金。
- 人的劳动力消耗所需要的代价是软件产品的主要成本。



成本的单位

货币单位

- 人民币元
- 美元
-



软件规模和成本的关系

- 规模是成本的主要因素，是成本估算的基础。
- 有了规模也就确定了成本。
- 例如，一个软件项目的规模是20人月，人力成本的平均值是2万/人月，则该项目的成本是40万元。



提问

() 是成本的主要因素，是成本估算的基础。

- A、计划
- B、规模 ✓
- C、风险
- D、利润



提问

下列不是软件项目规模单位的是（ ）。

- A、源代码长度（LOC）
- B、功能点（FP）
- C、人天、人月、人年
- D、小时 ✓

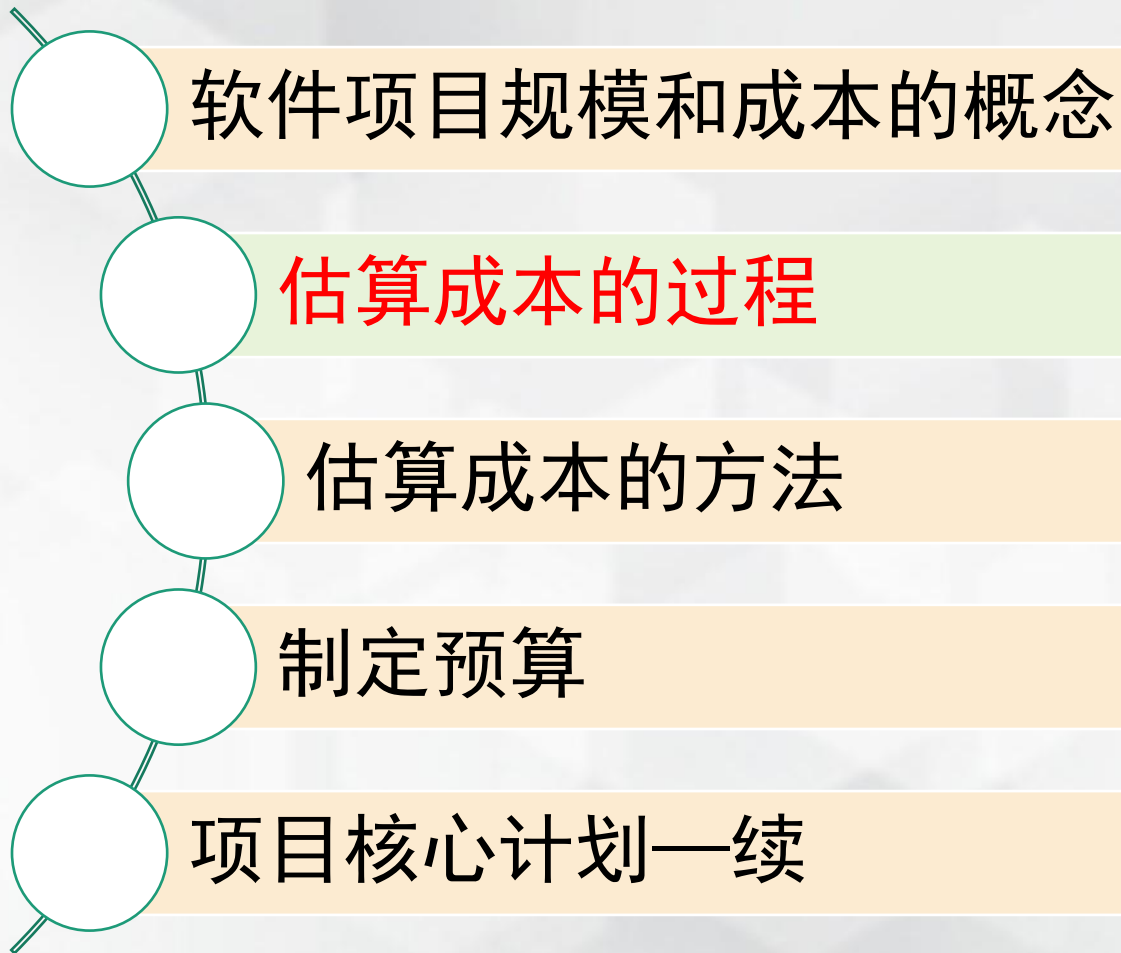


提问

软件项目的主要成本是人工成本。



本章内容





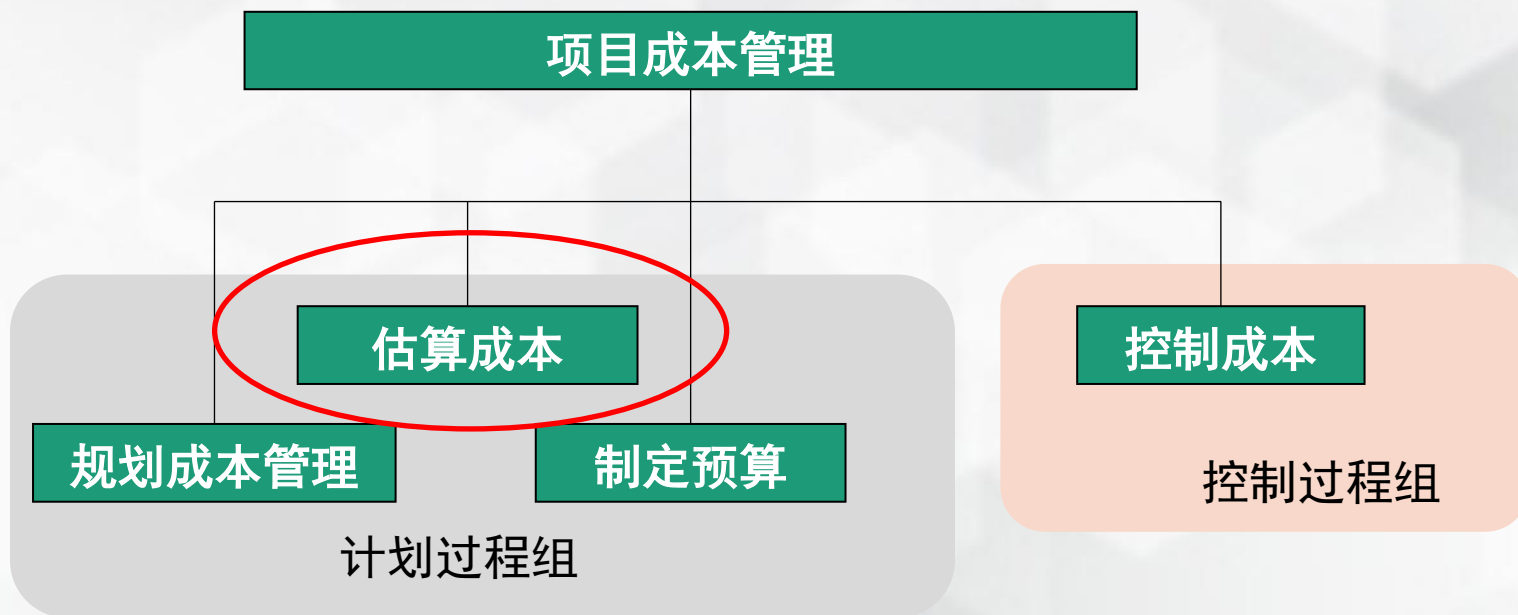
估算？

- 估算不是很准确的、是有误差的
- 经验（历史）数据非常重要
- 不要太迷信数学模型



什么是项目成本管理?

项目成本管理，就是为了在审批过的项目预算内完成项目，所进行的成本计划、估算、预算、监视和控制等一系列管理活动。





估算成本

估算成本就是，针对项目的整体预算，来回答
大概需要花费多少费用的过程。

输入	工具和方法	输出
1. 组织的环境因素 2. 组织的过程资产 3. 范围基线 4. 项目进度计划 5. 人力资源规划 6. 风险登记簿	1. 代码行、功能点 2. 类比估算法 3. 自下而上估算法 4. 参数模型估算法 5. 专家估算法 6. 储值分析 7. 项目成本估算软件	1. 活动成本估算 2. 活动成本估算详细资料 3. 风险登记簿（更新版）



估算成本

估算成本
大概需要

1. 组织的环境因素

市场条件、经济景气状况、资源成本费率、汇率等。

工，来回答

输入	工具和方法	输出
1. 组织的环境因素 2. 组织的过程资产 3. 范围基线 4. 项目进度计划 5. 人力资源规划 6. 风险登记簿	1. 代码行、功能点 2. 类比估算法 3. 自下而上估算法 4. 参数模型估算法 5. 专家估算法 6. 储值分析 7. 项目成本估算软件	1. 活动成本估算 2. 活动成本估算详细资料 3. 风险登记簿（更新版）



估算成本

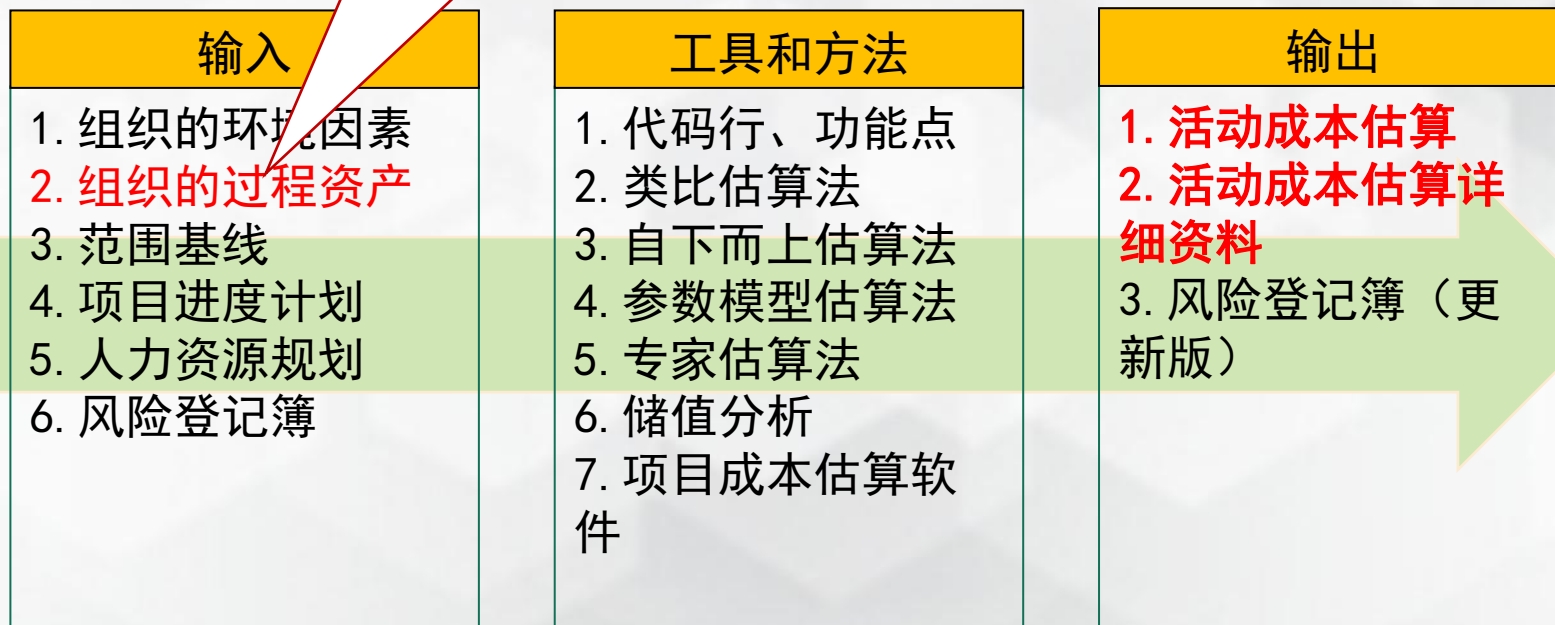
估算成本

大概需要

2. 组织的过程资产

成本估算模板、历史信息、经验教训等。

预算，来回答





估算成本

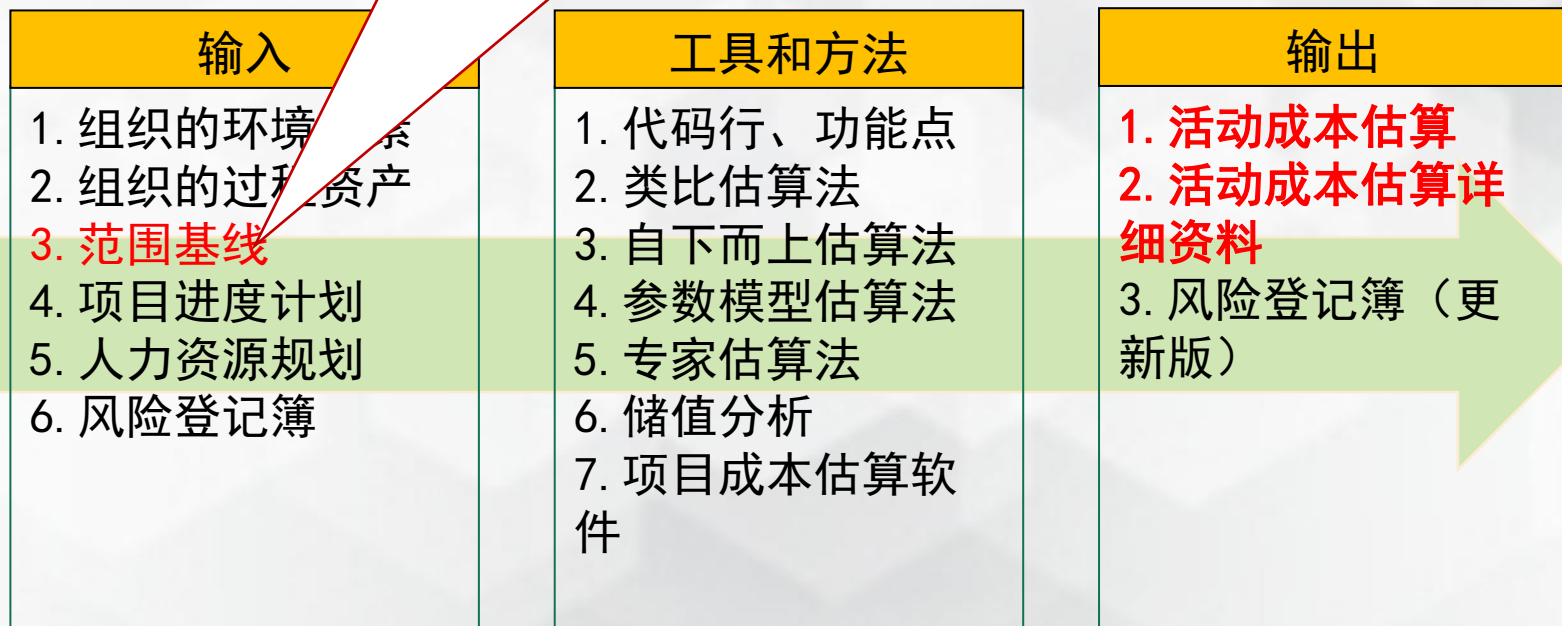
估算成本

大概需要

3. 范围基线

范围基线包括批准的需求规格说明书，WBS和WBS字典。这三点是成本估算的基础。**非常重要！**

，来回答





估算成本

估算成本

大概需要

4. 项目进度计划

它是项目进度管理的计划过程的输出。其中的活动资源估计（什么时候需要什么类型的资源，需要多少）是成本估算所需要的信息。

来回答

输入	工具和方法	输出
1. 组织的环境因素 2. 组织的过程资产 3. 范围基线 4. 项目进度计划 5. 人力资源规划 6. 风险登记簿	1. 代码行、功能点 2. 类比估算法 3. 自下而上估算法 4. 参数模型估算法 5. 专家估算法 6. 储值分析 7. 项目成本估算软件	1. 活动成本估算 2. 活动成本估算详细资料 3. 风险登记簿（更新版）



估算成本

估算成本

大概需要

5. 人力资源规划

它是项目人力资源管理中计划过程的输出。它所做出的人力资源计划是成本估算的重要输入。

来回答

输入	工具和方法	输出
1. 组织的环境 2. 组织的过程资产 3. 范围基线 4. 项目进度计划 5. 人力资源规划 6. 风险登记簿	1. 代码行、功能点 2. 类比估算法 3. 自下而上估算法 4. 参数模型估算法 5. 专家估算法 6. 储值分析 7. 项目成本估算软件	1. 活动成本估算 2. 活动成本估算详细资料 3. 风险登记簿（更新版）



估算成本

估算成本

大概需要

6. 风险登记簿

它是项目风险管理中计划过程的输出。通过审查风险登记簿，来考虑降低风险所需要的成本。

来回答

输入	工具和方法	输出
1. 组织的环境 2. 组织的过程 3. 范围基线 4. 项目进度计划 5. 人力资源规划 6. 风险登记簿	1. 代码行、功能点 2. 类比估算法 3. 自下而上估算法 4. 参数模型估算法 5. 专家估算法 6. 储值分析 7. 项目成本估算软件	1. 活动成本估算 2. 活动成本估算详细资料 3. 风险登记簿（更新版）



估算成本

估算成本就是，针对项目的整体预算，来回答
大概需要花费多少费用的过程。

输入	工具和方法	输出
1. 组织的环境因素 2. 组织的过程资产 3. 范围基线 4. 项目进度计划 5. 人力资源规划 6. 风险登记簿	1. 代码行、功能点 2. 类比估算法 3. 自下而上估算法 4. 参数模型估算法 5. 专家估算法 6. 储值分析 7. 项目成本估算软件	1. 活动成本估算 2. 活动成本估算详细资料 3. 风险登记簿（更新版）



估算成本的输出

1. 活动成本估算

- 是对完成项目工作可能需要成本的量化估算。
- 成本估算应该覆盖活动所使用的全部资源，包括：
 - 直接人工、材料、设备、服务、设施、信息技术；
 - 一些特殊的成本种类，如通货膨胀补贴或成本应急储备；
 - 如果间接成本也包括在项目中，则可在活动层次或更高层次上记列间接成本。



估算成本

估算成本就是，针对项目的整体预算，来回答
大概需要花费多少费用的过程。

输入	工具和方法	输出
1. 组织的环境因素 2. 组织的过程资产 3. 范围基线 4. 项目进度计划 5. 人力资源规划 6. 风险登记簿	1. 代码行、功能点 2. 类比估算法 3. 自下而上估算法 4. 参数模型估算法 5. 专家估算法 6. 储值分析 7. 项目成本估算软件	1. 活动成本估算 2. 活动成本估算详细资料 3. 风险登记簿（更新版）



估算成本的输出

2. 活动成本估算详细资料

- 支持性文件应完整、清晰地描述成本估算是如何得到的。
- 它包括如下一些信息：
 - 估算依据（估算是如何编制的）
 - 估算的假设条件
 - 各种已知的制约因素
 - 对估算区间的说明（例如：10000美金±10%）
 - 估算结果置信水平的说明。



估算成本

估算成本就是，针对项目的整体预算，来回答
大概需要花费多少费用的过程。

输入	工具和方法	输出
1. 组织的环境因素 2. 组织的过程资产 3. 范围基线 4. 项目进度计划 5. 人力资源规划 6. 风险登记簿	1. 代码行、功能点 2. 类比估算法 3. 自下而上估算法 4. 参数模型估算法 5. 专家估算法 6. 储值分析 7. 项目成本估算软件	1. 活动成本估算 2. 活动成本估算详细资料 3. 风险登记簿（更新版）



估算成本的工具和方法

6. 储值分析

- 为了应对成本的不确定性，成本估算中可以包括应急储备（有时称为“应急补贴”）。
- 应急储备可以是成本估算值的某个百分比、某个固定值，或者通过定量分析得到。
- 随着项目信息越来越明确，可以动用、减少或取消应急储备。
- 应该在项目成本文件中清楚地列出应急储备。
- 应急储备是资金需求的一部分。



估算成本

估算成本就是，针对项目的整体预算，来回答大概需要花费多少费用的过程。

输入	工具和方法	输出
1. 组织的环境因素 2. 组织的过程资产 3. 范围基线 4. 项目进度计划 5. 人力资源规划 6. 风险登记簿	1. 代码行、功能点 2. 类比估算法 3. 自下而上估算法 4. 参数模型估算法 5. 专家估算法 6. 储值分析 7. 项目成本估算软件	1. 活动成本估算 2. 活动成本估算详细资料 3. 风险登记簿（更新版）



估算成本的工具和方法

7. 项目成本估算软件

- 项目成本估算软件（如：成本估算应用软件、电子表格软件、模拟和统计软件等）对辅助成本估算的作用，已经得到广泛的认可。
- 这些工具能够简化某些成本估算技术的使用，使人们能够快速考虑多个成本估算方案。



成本估算的说明

- 预测所需要的总成本的过程。
- 是一种量化的结果。
- 可以有一些误差。
- 成本估算不同于项目定价。
- 贯穿于软件的整个生命周期

项目启动阶段可以得到一个粗略的估算（如 $\pm 50\%$ ），
随着信息越来越详细，估算误差可以缩小到 $\pm 10\%$ 。



成本的种类

在做成本估算的时候，我们还需要考虑以下两类成本。

➤ 直接成本

➤ 间接成本



直接成本

与具体项目相关的成本，例如：

- 人员的工资、加班费
- 项目使用的材料费
- 外包成本、外购成本等



间接成本

- 不能具体到某个项目中的成本
- 可以分摊到各个具体项目中的成本，例如：
 - 培训
 - 房租水电
 - 员工福利
 - 市场费用
 - 管理费
 - 其他等等



提问

下面关于估算的说法，错误的是（ ）。

- A、估算是有误差的
- B、估算时不要太迷信数学模型
- C、经验对于估算来说不重要 ✓
- D、历史数据对于估算来说非常重要

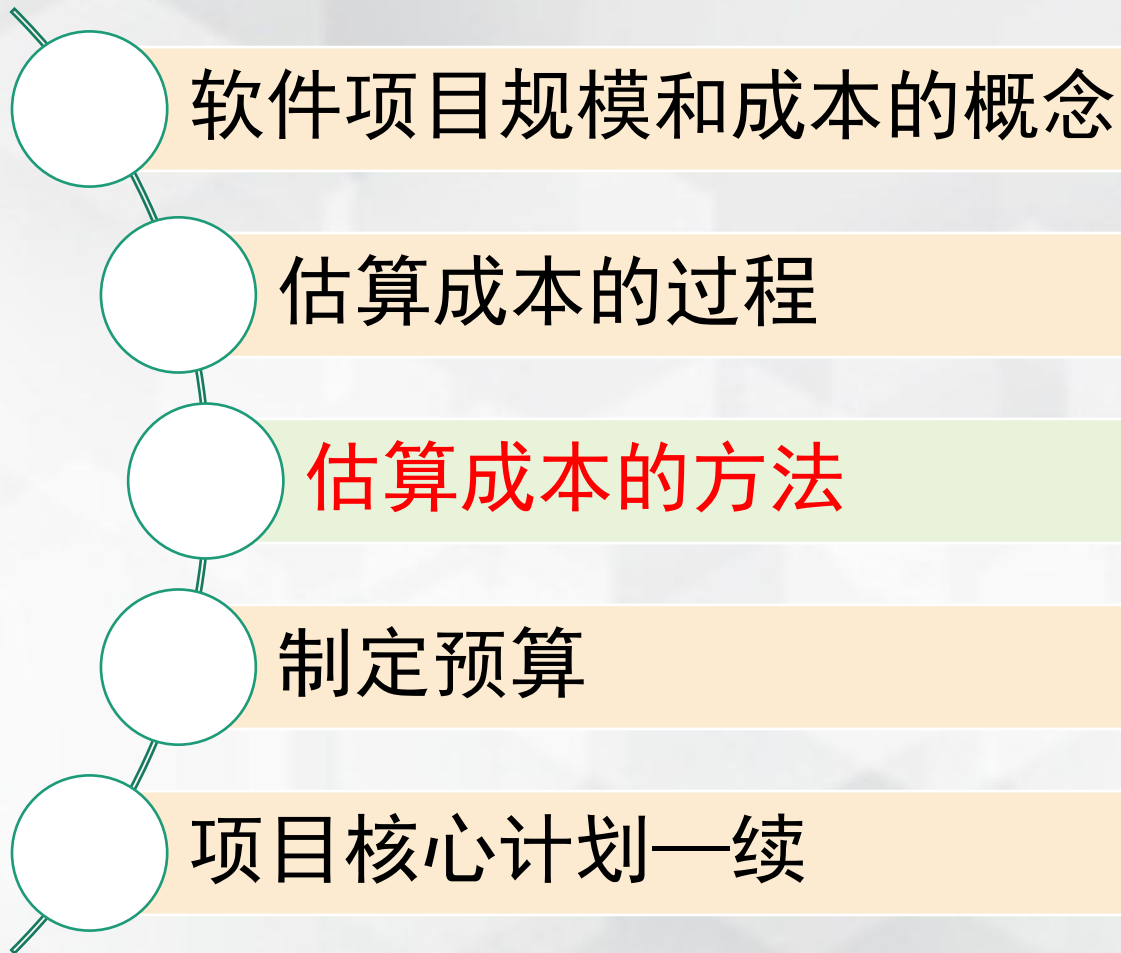


提问

软件项目成本包括直接成本和间接成本，一般而言，项目人力成本归属于直接成本。



本章内容





估算成本的方法

1. 代码行、功能点

2. 类比估算法

3. 自下而上估算法

4. 参数模型估算法

5. 专家估算法



估算成本的方法

1. 代码行、功能点

2. 类比估算法

3. 自下而上估算法

4. 参数模型估算法

5. 专家估算法



代码行 (LOC) 技术

是一种软件规模估算方法。从软件程序量的角度来估算项目规模。

➤ 要求

- 功能分解的足够详细
- 有一定的经验数据（类比和经验方法）
- 与具体的编程语言有关



代码行技术的优点

- 代码是所有软件开发项目都有的“产品”。
- 很容易计算代码行数。



代码行技术的缺点

- 对代码行没有公认的可接受的标准定义。
- 代码行数量依赖于所用的编程语言和个人的编程风格。
- 在项目早期，需求不稳定、设计不成熟、实现方法不确定的情况下很难准确地估算代码量。
- 代码行强调编码的工作量，只是项目开发阶段的一部分。



功能点 (Function Point)

- 用系统的**功能数量**来测量其规模。
- 与实现产品所使用的语言和技术没有关系的。
- 估算过程
 - 产品评估
 - 内部基本功能
 - 外部基本功能
 - 根据技术复杂度因子，对它们进行加权和量化。



功能点计算公式

$$FP = UFC * TCF$$

其中：

- UFC (Unadjusted Function Component) : 未调整功能点计数
- TCF (Technical Complexity Factor) : 技术复杂度因子 (调整系数)



功能点计算公式

$$FP = UFC * TCF$$

其中：

- UFC (Unadjusted Function Component) : 未调整功能点计数
- TCF (Technical Complexity Factor) : 技术复杂度因子 (调整系数)



UFC-未调整功能点计数

外部输入

(EI、External Input)

- 给软件提供面向应用的数据项。如界面、对话框、表单

外部输出

(EO、External Output)

- 向用户提供面向应用的信息。如报表、错误信息

外部查询

(EQ、External Query)

- 一次联机输入，导致软件以联机输出方式产生的某种即时响应

外部接口文件

(EIF、External Interface File)

- 用户可以识别的一组逻辑相关数据，只读文件。
- 存在于应用的外部，由另一个应用维护。

内部逻辑文件

(ILF、Internal Logic File)

- 用户可以识别的一组逻辑相关数据。
- 存在于应用的边界之内，通过外部输入维护。



UFC-未调整功能点计数

未调整功能计数项的复杂度等级：

计数项	复杂度权重		
	简单	一般	复杂
外部输入	3	4	6
外部输出	4	5	7
外部查询	3	4	6
外部接口文件	5	7	10
内部逻辑文件	7	10	15



UFC计算实例

	功能点		
计数项	简单	一般	复杂
外部输入	6 * 3	2 * 4	3 * 6
外部输出	7 * 4	7 * 5	0 * 7
外部查询	0 * 3	2 * 4	4 * 6
外部接口文件	5 * 5	2 * 7	3 * 10
内部逻辑文件	9 * 7	0 * 10	2 * 15
总计	134	65	102
UFC	301		



功能点计算公式

$$FP = UFC * TCF$$

其中：

➤ UFC (Unadjusted Function Component) : 未调整功能点计数

➤ TCF (Technical Complexity Factor) : 技术复杂度因子 (调整系数)



TCF-技术复杂度因子

$$TCF = 0.65 + 0.01 (\text{sum}(F_i))$$

$F_i: 0 \sim 5$ 、 $TCF: 0.65 \sim 1.35$

技术复杂度因子			
F1	可靠的备份和恢复	F2	数据通信
F3	分布式函数	F4	性能
F5	大量使用的配置	F6	联机数据输入
F7	操作简单性	F8	在线升级
F9	复杂界面	F10	复杂数据处理
F11	重复使用性	F12	安装简易性
F13	多重站点	F14	易于修改

技术复杂度因子取决于上述14个通用系统特征。



调整系数 F_i 的取值范围

调整系数	描述
0	不存在或者没有影响
1	不显著的影响
2	相当的影响
3	平均的影响
4	显著的影响
5	强大的影响



功能点计算实例

$$\text{FP} = \text{UFC} * \text{TCF}$$

➤ $\text{UFC} = 301$

➤ $\text{TCF} = 0.65 + 0.01(14 * 3) = 1.07$

$$\text{FP} = 301 * 1.07 = 322$$



功能点与代码行的转换关系

语言	代码行/FP
Assembly	320
C	150
COBOL	105
FORTRAN	105
PASCAL	91
ADA	71
PL/1	65
PROLOG/LISP	64
SMALLTALK	21
SPREADSHEET	6



估算成本的方法

1. 代码行、功能点

2. 类比估算法

3. 自下而上估算法

4. 参数模型估算法

5. 专家估算法



类比估算法

- 估算人员根据以往的完成类似项目所消耗的总成本（或总工作量），来推算将要开发的软件的总成本（或总工作量），然后按比例将它分配到各个开发任务单元中。
- 可以理解为是一种**自上而下**的估算形式。



类比估算法的使用场景

- 有类似的历史项目数据
- 信息不足（要求不是非常精确）的时候
- 在招标或者合同文本准备阶段



类比估算法的特点

- 简单易行、成本低、耗时少。
- 具有一定的局限性。
- 准确性差，可能导致项目出现困难。



估算成本的方法

1. 代码行、功能点

2. 类比估算法

3. 自下而上估算法

4. 参数模型估算法

5. 专家估算法



自下而上估算法

利用**任务分解结构（WBS）**，对各个具体活动进行详细的成本估算，然后将结果累加起来得出项目总成本。



自下而上估算法的使用场景

- 项目开始以后，或者WBS已经确定。
- 需要进行较为精确的成本估算的时候。



自下而上估算法的特点

- 这种方法相对比较准确，它的准确度来源于每个任务的估算情况。
- 比较费时，估算本身也需要成本支持。
- 可能发生虚报现象。



自下而上估算法举例

活动内容	人工费	材料费	间接费	费用合计
活动A	100	50	15	165
活动B	20	0	2	22
...	...	...	...	...
活动Z	150	30	18	198
合计	2500	500	300	3300



估算成本的方法

1. 代码行、功能点

2. 类比估算法

3. 自下而上估算法

4. 参数模型估算法

5. 专家估算法



参数模型估算法

一种使用项目特性参数建立**数学模型**来估算成本的方法，是一种统计技术。



参数模型估算法的使用场景

- 存在成熟的项目估算模型。
- 应该具有良好的基础数据为基础。



参数模型估算法的特点

- 比较简单，而且也比较准确。
- 如果模型选择不当或者基础数据不准，也会导致偏差。
- 目前没有任何一种模型或者方法能适应所有项目。



参数模型估算法的思路

- 提供工作量(规模)的直接估算。
- 通过过去项目数据, 进行回归分析, 得出的回归模型。



参数模型估算法的整体公式

$$E = a + b \times S^c$$

其中：

E: 以人月表示的工作量

a, b, c: 经验导出的系数

S: 主要的输入参数 (通常是LOC, FP等)



基于LOC的估算模型公式

➤Walston-Felix (IBM) 模型

$$E = 5.2 \times (\text{KLOC})^{0.91}$$

➤Bailey-Basili模型

$$E = 5.5 + 0.73 \times (\text{KLOC})^{1.16}$$

➤Boehm (基本COCOMO) 模型

$$E = 3.2 \times (\text{KLOC})^{1.05}$$

➤Doty模型

$$E = 5.288 \times (\text{KLOC})^{1.047}$$



基于FP的估算模型公式

➤ Albrecht and Gaffney模型

$$E = -12.39 + 0.0545 \times FP$$

➤ Matson, Barnett模型

$$E = 585.7 + 15.12 \times FP$$



建议掌握的两个模型

- IBM模型 (Walston-Felix)
- 基本COCOMO模型 (Boehm)



IBM模型

1977年，IBM的Walston和Felix提出了如下的估算公式

$E = 5.2 \times L^{0.91}$ ，L是源代码行数（以KLOC计），E是工作量（以PM计）

$D = 4.1 \times L^{0.36}$ ，D是项目持续时间（以月计）

$S = 0.54 \times E^{0.6}$ ，S是人员需要量（以人计）

$DOC = 49 \times L^{1.01}$ ，DOC是文档数量（以页计）



IBM模型举例

采用java 完成项目，366功能点，则：

$$L = 366 \times 46 = 16386 \text{行} = 16.386 \text{KLOC}$$

$$E = 5.2 \times L^{0.91} = 5.2 \times 16.386^{0.91} \approx 66 \text{人月}$$

$$DOC = 49 \times L^{1.01} = 49 \times 16.386^{1.01} \approx 826 \text{页}$$



COCOMO模型

- 英文名称: **Constructive Cost Model**
- 中文名称: **结构化成本模型**
- 是世界上应用最广泛的参数型软件**成本估算**和**历时估算**模型;
- 由Barry Boehm (巴利·玻姆) 在1981年开发的;
- 它是一个基于过去的软件项目开发数据得到的一个统计模型。



COCOMO模型的发展

COCOMO 81

- 最初的COCOMO模型。根据TRW公司的（主要从事军需、精密仪器软件开发）实绩数据作成的。

COCOMO II

- 还是由转入南加州大学的Barry Boehm提出。在COCOMO81的基础上，对应迭代开发模型、面向对象、80-90年代的开发工具和开发环境的变化。分别于1998年、1999年、2000年对模型进行了更新。

模型系列

- 目前，包含了软件成本模型、软件扩展与其他独立估算模型3大类。



模型类别：

- 基本COCOMO
- 中等COCOMO
- 高级COCOMO

项目模式：

- 有机：Organic
- 嵌入式：Embedded
- 半有机：Semi detached



COCOMO81的模型类别

基本COCOMO

- 静态单变量模型。
- 在项目信息极少的情况下使用。

中等COCOMO

- 在基本COCOMO模型基础上考虑影响因素，调整模型。
- 在需求确定以后使用。

高级COCOMO

- 在中等COCOMO模型基础上考虑各个步骤的影响。
- 在设计完成后使用。



COCOMO81的项目模式

有机:Organic

- 各类应用程序，例如数据处理、科学计算等。
- 受硬件的约束比较小，程序的规模不是很大。

嵌入式:Embedded

- 各类系统程序，例如实时处理、控制程序等。
- 紧密联系的硬件、软件 and 操作的限制条件下运行，软件规模任意。

半有机（半嵌入式）:Semidetached

- 各类实用程序，例如编译程序、分析程序等。
- 介于上述两种模式之间，规模和复杂度都属于中等或者更高。



基本COCOMO模型

$$E = a \times (KL0C)^b$$

其中：

E是所需的人力（人月）

KL0C是交付的代码行

a，b是依赖于项目模式的参数



基本COCOMO模型

项目模式	a	b
有机	2.4	1.05
半有机	3.0	1.12
嵌入式	3.6	1.2



基本COCOMO模型举例

一个33.3KL0C的软件开发项目，属于中等规模、半有机型的项目，采用基本COCOMO模型：

$$a=3.0, b=1.12。$$

$$E = 3.0 \times \text{KL0C}^{1.12} = 3.0 \times 33.3^{1.12} = 152\text{PM}$$



中等COCOMO模型

$$E = a \times (KLOC)^b \times \text{乘法因子}$$

a、b是系数

乘法因子是根据成本驱动因子打分的结果，
是对公式的校正系数。



中等COCOMO模型

项目模式	a	b
有机	3.2	1.05
半有机	3.0	1.12
嵌入式	2.8	1.2



中等COCOMO模型的成本驱动因子

- 产品属性
- 平台属性
- 人员属性
- 过程属性



中等COCOMO模型的成本驱动因子

P118

Cost drivers	Rating levels						Remark
	Very low	Low	Nominal	High	Very high	Extra high	
Product attributes							
RELY	0.75	0.88	1.00	1.15	1.40	1.65	Required software reliability
DATA		0.94	1.00	1.08	1.16		Database size
CPLX	0.70	0.85	1.00	1.15	1.30		Product complexity
Computer attributes							
TIME			1.00	1.11	1.30	1.66	Execution time constraints
STOR			1.00	1.06	1.21	1.56	Main storage constraints
VIRT		0.87	1.00	1.15	1.30		Virtual machine volatility
TURN		0.87	1.00	1.07	1.15		Computer turnaround time
Personnel attributes							
ACAP	1.46	1.19	1.00	0.86	0.71	0.70	Analyst capability
AEXP	1.29	1.13	1.00	0.91	0.82		Applications experience
PCAP	1.42	1.17	1.00	0.86			Programming capability
VEXP	1.21	1.10	1.00	0.90			Virtual machine experience
LEXP	1.14	1.07	1.00	0.95			Language experience
Process attributes							
MODP	1.24	1.10	1.00	0.91	0.82	0.83	Use of modern programming practices
TOOL	1.24	1.10	1.00	0.91			Use of software tools
SCED	1.23	1.08	1.00	1.04	1.10		Required development schedule



中等COCOMO模型的成本驱动因子

➤每个属性 F_i 的取值范围为：

很低、低、正常、高、很高、极高，共六级。

正常情况下： $F_i=1$ 。

➤当每个 F_i 的值选定后，成本驱动因子的计算如下：

$$\text{乘法因子} = F_1 * F_2 * \dots * F_i * \dots * F_n$$



中等COCOMO模型举例

一个33.3KLOC的软件开发项目，属于中等规模、半有机型的项目，采用中等COCOMO模型：

$a=3.0$ ， $b=1.12$ 。

成本驱动因子 $=0.70*0.85*1\cdots\cdots*1.15=1.09$

$$E = 3.0 \times \text{KLOC}^{1.12} \times F =$$

$$3.0 \times 33.3^{1.12} \times 1.09 = 166\text{PM}$$



高级COCOMO模型

- 将项目分解为一系列的子系统或者子模型。
- 在一组子模型的基础上更加精确地调整一个模型的属性，增加了与开发阶段相关的工作量乘数，更准确地反映成本驱动因子对工作量阶段分布的影响。



高级COCOMO模型

阶段性差异

成本驱动因子		级别					
		很低	低	正常	高	很高	极高
应用 经验	需求设计阶段	1.40	1.20	1	0.87	0.75	
	详细设计阶段	1.30	1.15	1	0.9	0.80	
	编码和单元测试阶段	1.25	1.10	1	0.92	0.85	
	集成测试阶段	1.25	1.10	1	0.92	0.85	



- 在改进和发展COCOMO81基础上，于1995年提出。
- 给出了三个层次的软件开发工作量估算模型
 - 应用系统组成模型
 - 用于估算构件原型的工作量
 - 适用于项目规划阶段
 - 早期设计模型
 - 适用于体系结构设计阶段
 - 用于信息还不足以支持详细设计的细粒度估算阶段
 - 后体系结构模型
 - 适用于完成体系结构设计之后的软件开发阶段
 - 基于源代码行/功能点、以及17个工作量乘数因子
 - 用于完成顶层设计和获取详细项目信息阶段



在COCOMO II 中，定义了计算规模、工作量、开发历时的基本公式。



COCOMO II 的规模估算

- 规模的估算不属于COCOMO的范畴，因此需要用别的方法来计算规模；
- 规模的单位是「KSLOC（千代码行数）」；
- 规模的计算根据新开发的量、现有的软件再利用或改造的量、自动生成或自动变换的量等组合来计算；
- 以下是其计算规模的公式。

基本式: 規模

$$\begin{aligned}
 \text{要件の変動性} & \quad \text{新規開発規模} & \quad \text{等価コード行数} \\
 \text{Size} &= \left(1 + \frac{\text{REVL}}{100}\right) \times (\text{NewKSLOC} + \text{EquivalentKSLOC}) \\
 \text{自動変換されるコードの割合} & \\
 \text{EquivalentKSLOC} &= \text{AdaptedKSLOC} \times \text{AAM} \times \left(1 - \frac{\text{AT}}{100}\right) \\
 \text{適応調整要因} & \quad \text{適応(改造)コード行数} & \quad \text{再利用ソフトウェア修正} \\
 \text{AAF} \leq 50 \text{ のとき,} & \\
 \text{AAM} &= \left\{ \text{AA} + \text{AAF} \times \left(1 + \left[0.02 \times \text{SU} \times \text{UNFM}\right]\right) \right\} / 100 \\
 \text{改修有効性調査} & \quad \text{ソフトウェア理解度} & \quad \text{プログラマ非習熟度} \\
 \text{AAF} > 50 \text{ のとき,} & \\
 \text{AAM} &= \left\{ \text{AA} + \text{AAF} + (\text{SU} \times \text{UNFM}) \right\} / 100 \\
 \text{変更が必要な設計・コードの割合} & \quad \text{結合・テストに必要な工数の割合} \\
 \text{AAF} &= (0.4 \times \text{DM}) + (0.3 \times \text{CM}) + (0.3 \times \text{IM})
 \end{aligned}$$

基本式: 規模 つづき

- ❖ *Size*: 規模
- ❖ *NewKSLOC*: 新規開発規模 (単位:KSLOC)
- ❖ *EquivalentKSLOC*: 等価コード行数(単位:KSLOC)
 - 既存のソフトウェアコード(対象システムに統合されていない)を再利用又は修正して利用する場合に必要なになる
- ❖ *AdaptedKSLOC*: 適応(改造)ソースコード規模 (単位:KSLOC)
- ❖ *AA*: 改修有効性調査
 - 再利用ソフトウェアの評価などのための増加度(%) (0~8%)
- ❖ *AAF*: 適応調整要因
 - 対象システムに結合する、既存ソフトウェアの変更による工数への影響を反映するために *DM*, *CM*, *IM* の3つの要因を予測した結果で表す
- ❖ *AAM*: 再利用ソフトウェア修正を表す変数
- ❖ *AT*: 自動変換されるコードの割合(%)

基本式：規模 つづき

- ❖ *DM*: 再利用・適応されるソフトウェアで、変更が必要な設計の割合(%)
- ❖ *CM*: 再利用・適応されるソフトウェアで、変更が必要なコードの割合(%)
- ❖ *IM*: 適応されたソフトウェアをプロダクト全体に対して結合するため、及び、結合結果のソフトウェアをテストするための工数を同様のサイズのソフトウェアに対する結合とテストに必要な平均的な工数と比較しての比率(%)
- ❖ *REVL*: 要件の変動性
 - 見積もり以降の要件の追加や変更により廃棄されるコードの割合で、開発後稼働するソフトウェアのコード行数に対する比率(%)
- ❖ *SU*: ソフトウェア理解度
 - 再利用ソフトウェア理解のための増加度(%)
 - VeryLow(50%)~VeryHigh(50%)の5段階
- ❖ *UNFM*: プログラマ非習熟度
 - プログラマが再利用ソフトウェアに精通していないことによる増加乗数 (0~1の間で評価)
 - 完全に精通(0.0)~全く馴染み無し(1.0)の6段階



COCOMO II 的工作量估算

- 工作量的估算是COCOMO的主要工作；
- 工作量根据规模来估算；
- 工作量的单位是**人月**；
- 在实施工作量估算的时候，根据估算时候的工程状况，选择Early Design Model（**早期设计模型**）或Post-Architecture Model（**后体系结构模型**）。



COCOMO II の作業量估算

基本式: 工数

標準工数

$$PM_{NS} = A \times Size^E \times \prod_{i=1}^n EM_i$$

定数

コストドライバ

工数

$$PM = A \times Size^E \times \prod_{i=1}^n EM_i + PM_{AUTO}$$

規模指数

$$E = B + 0.01 \times \sum_{j=1}^5 SF_j$$

規模指数の基底値

規模要因

自動変換されるコードの割合

自動変換活動工数

$$PM_{AUTO} = \frac{\text{AdaptedKSLOC} \times \left(\frac{AT}{100} \right)}{ATPROD}$$

適応(改造)コード行数

自動変換による作業の生産性

基本式：工数 つづき

- ❖ PM_{NS} ：標準工数(人月)
 - 標準的な開発期間の場合で、自動変換活動工数 PM_{AUTO} を含まない予測工数
 - コストドライバの SCED の値を除いて算出する
- ❖ $Size$ ：規模
- ❖ E ：規模指数
- ❖ SF ：規模要因 (5個)
- ❖ EM ：コストドライバ(工数調整係数)を表す変数
 - Early Designは7個、Post-Architectureは17個
- ❖ PM ：開発工数(人月)
 - 開発期間への要求を考慮し、自動変換活動工数も含めた全体工数
- ❖ PM_{AUTO} ：自動変換の活動の工数(人月)

基本式：工数 つづき

- ❖ A : 開発規模が開発工数に与える影響の大きさを表す定数
 - 較正 (calibration: モデル化手法に含まれる) によって求める
- ❖ AT : 自動変換されるコードの割合(%)
 - 再利用・適応する既存ソフトウェアを自動変換することによって適応先のソフトウェアに適合させることができるコードの割合(%)
- ❖ $ATPROD$: 自動変換による作業の生産性
- ❖ B : 工数予測式の規模指数の基底値
 - 較正によって求める
- ❖ $AdaptedKSLOC$: 適応(改造)ソースコード規模.
 - ホワイトボックスとして扱われ、利用するために修正されうる既存コード



估算成本

估算成本就是，针对项目的整体预算，来回答大概需要花费多少费用的过程。

输入	工具和方法	输出
1. 组织的环境因素 2. 组织的过程资产 3. 范围基线 4. 项目进度计划 5. 人力资源规划 6. 风险登记簿	1. 代码行、功能点 2. 类比估算法 3. 自下而上估算法 4. 参数模型估算法 5. 专家估算法 6. 储值分析 7. 项目成本估算软件	1. 活动成本估算 2. 活动成本估算详细资料 3. 风险登记簿（更新版）



估算成本的方法

1. 代码行、功能点

2. 类比估算法

3. 自下而上估算法

4. 参数模型估算法

5. 专家估算法



专家估算法

由多位专家进行成本估算，取得多个估算值，最后得出综合的估算值。



专家估算法的步骤

- 组织者发给每位专家一份软件系统的规格说明和一张记录估算值的表格，请他们估算。
- 专家详细研究软件规格说明后，对该软件提出3个规模的估算值。
 - 最小 O_i
 - 最可能的 M_i
 - 最大 P_i
- 组织者为专家的表格中的答复进行整理。
- 计算每位专家的 $E_i = (O_i + 4M_i + P_i) / 6$



专家估算法的步骤

- 综合结果后： $E = (E_1 + E_2 + \dots + E_n) / n$ 。n表示n个专家。
- 再组织专家无记名填表格，比较估算差，并查找原因。
- 如果各个专家的估算差异超出规定的范围（例如：15%），则需重复上述过程，最终可以获得一个多数专家共识的软件规模。



专家估算法举例

某多媒体信息查询系统的专家估算。

➤ **专家1:** 1, 8, 9 $\Rightarrow (1+4*8+9)/6=7$ (万元)

➤ **专家2:** 4, 6, 8 $\Rightarrow (4+4*6+8)/6=6$ (万元)

估算结果 $= (6+7)/2=6.5$ (万元)



成本估算法小结

项目初期

- 类比法
- 专家估算法

计划阶段

- 自下而上
- 参数估算法

实施阶段（包括变更发生）

- 自下而上
- 参数估算法



提问

在项目初期，一般采用的成本估算方法是

类比估算法

。



提问

常见的成本估算方法不包括（ D ）。

A、代码行

B、功能点

C、类比法

D、关键路径法



提问

下列不是UFC的功能计数项是（ C ）。

- A、外部输出
- B、外部接口文件
- C、内部输出
- D、内部逻辑文件



提问

估算成本的基本方法不包括（ D ）。

A、代码行、功能点

B、参数估算法

C、专家估算法

D、函数估算法



提问

在项目初期，进行竞标合同时，一般采用的成本估算方法是（ B ）。

- A、参数估算法
- B、类比估算法
- C、专家估算法
- D、功能点估算法



实用的软件项目成本估算方法

这是一种自下而上的方法，步骤如下：

1. 对任务进行分解（WBS）：1, 2, ..., i ... n
2. 估算每个活动/工作包的成本 E_i
3. 总估算成本= $E_1+E_2+\dots+E_i+\dots+E_n$
4. 估算成本 = 直接成本 + 间接成本
5. 项目总报价=总估算成本+项目风险利润

其中，项目风险利润 = 利润 + 风险基金 + 税



实用的软件项目成本估算方法

这是一种自下而上的方法，步骤如下：

1. 对任务进行分解（WBS）： $1, 2, \dots, i \dots n$
2. 估算每个活动/工作包的成本 E_i
3. 总估算成本= $E_1 + E_2 + \dots + E_i + \dots + E_n$
4. 项目总报价=总估算成本+项目风险利润

其中，项目风险利润 = 利润 + 风险基金 + 税



估算每个任务/工作包的成本

有两种方法：

➤ 直接估算成本 E_i

➤ 先估算工作量 Q_i ，然后估算成本 $E_i = Q_i * \text{人力成本参数}$ 。估算工作量的两种方法如下：

➤ 唯一估计值： $Q_i = \text{Avg}$

➤ PERT算法： $Q_i = (P + 4M + O) / 6$



估算每个任务/工作包的成本

有两种方法：

➤ 直接估算成本 E_i

➤ 先估算工作量 Q_i ，然后估算成本 $E_i = Q_i * \text{人力成本参数}$ 。估算工作量的两种方法如下：

➤ 唯一估计值： $Q_i = \text{Avg}$

➤ PERT算法： $Q_i = (P + 4M + O) / 6$

工作量又如何去考虑呢？



估算每个任务/工作包的成本

➤开发工作量：

- $\text{Scale}(\text{Dev})$ (单位：人月)

➤管理、质量工作量：

- $\text{Scale}(\text{Mgn}) = a * \text{Scale}(\text{Dev})$
- 其中 a 为比例系数。例如：20%~25%。

➤工作量 = $\text{Scale}(\text{Dev}) + a * \text{Scale}(\text{Dev})$



估算每个任务/工作包的成本

➤ 直接成本 = 工作量*人力成本参数

➤ 直接成本包括

➤ 开发成本

➤ 管理成本

➤ 质量成本

例如：假设人力成本参数=2万/人月，工作量为30人月的项目的直接成本是60万。



估算每个任务/工作包的成本

总估算成本 = 直接成本 + 间接成本

间接成本的估算：

- 按照企业模型直接估算。
- 简易算法：
 - 间接成本=直接成本*间接成本系数
 - 间接成本=工作量*人力成本参数*间接成本系数
 - 例如：设置间接成本系数 = 0.3



估算每个任务/工作包的成本

- 总估算成本=直接成本+间接成本
 - 总估算成本=直接成本+直接成本*间接成本系数
 - 总估算成本=直接成本*(1+间接成本系数)
 - 总估算成本=工作量*人力成本参数*(1+间接成本系数)
 - 成本系数=人力成本参数* (1+间接成本系数)
-
- 简易算法:
 - 估算成本 = 工作量*成本系数
 - 例如：成本系数 = 3万/人月



实用的软件项目成本估算方法

这是一种自下而上的方法，步骤如下：

1. 对任务进行分解（WBS）： $1, 2, \dots, i \dots n$
2. 估算每个活动/工作包的成本 E_i
3. 总估算成本= $E_1 + E_2 + \dots + E_i + \dots + E_n$
4. 项目总报价=总估算成本+项目风险利润

其中，项目风险利润 = 利润 + 风险基金 + 税



实用的软件项目成本估算方法

这是一种自下而上的方法，步骤如下：

1. 对任务进行分解（WBS）： $1, 2, \dots, i \dots n$
2. 估算每个活动/工作包的成本 E_i
3. 总估算成本= $E_1 + E_2 + \dots + E_i + \dots + E_n$
4. 项目总报价=总估算成本+项目风险利润

其中，项目风险利润 = 利润 + 风险基金 + 税



项目总报价

项目总报价 = 总估算成本 + 风险利润

其中风险利润包括：

- 项目利润 = 总估算成本*a%
- 风险基金 = 总估算成本*b%
- 项目税费 = 总估算成本*c% (例如:c为5.5左右)

项目总报价 = 总估算成本 + (a+b+c)%*总估算成本



成本估算的准确度

类型	准确度	说明
量级估算: 合同前 Order of magnitude	-25%~+75%	概念和启动阶段决策
预算估算: 合同期 Budget	-10%~+25%	编制初步计划
确定性估算: WBS后 Definitive	-5%~+10%	工作分解后的详细计划



成本估算不准的原因

- 基础数据不足
- 缺乏经验的估算人员
- 签约前后不连贯
- 低劣的估算技术
- 估算对需求的敏感程度



避免不准确成本估算的措施

1. 避免无准备的估算。
2. 留出估算的时间，并做好准备。
3. 使用以前的项目数据。
4. 使用开发人员提供的数据为基础估算。
5. 分类法估算（使用多种估算方法分别估算，然后比较结果）。
6. 详细的较低层次上的估算。
7. 使用软件估算工具。



估算结果的表达方式

➤ 加减限定

6个人月, +3人月, -2人月

➤ 范围

5-9人月



估算结果的表达方式

估算结果：6个人月、+3人月、-2人月

+1人月：延迟交付转换子系统

-1人月：新成员的工作效率高

+1人月：采用的新工具没有预计的好

-1人月：采用的新工具比预期的好

+0.5人月：员工病事假

+0.5人月：低估规模



估算结果的表达方式

估算结果：100人月~150人月

情况	估算
最佳情况	100人月
计划情况	110人月
最差情况	150人月



《软件工程中的真相与假象》

Glass在《软件工程中的真相与假象》一书中，总结了55个真相和10个假象，其中直接和估算有关的有7个真相和一个假象。



《软件工程中的真相与假象》

- 真相8: 造成软件项目失控最普遍的原因之一就是软件估算不足。
- 真相9: 往往在错误的时间，甚至在还没有定义需求也没有理解问题之前，就进行软件估算。
- 真相10: 往往由错误的人员，即不是由软件开发人员或者项目经理，而是由高层管理或者市场营销，进行项目估算。



《软件工程中的真相与假象》

- **真相11:** 由错误的人员，在错误的时间做出的软件估算往往是错误的，也很少随着项目的进行做必要的调整。
- **真相12:** 虽然软件估算会发生一些错误，但是人们不去关心产生错误的原因，却还在努力按照错误估算的时间进度执行。
- **真相13:** 管理人员和技术人员对软件估算的认识是完全隔绝的，因此也难以达到一致的目标。

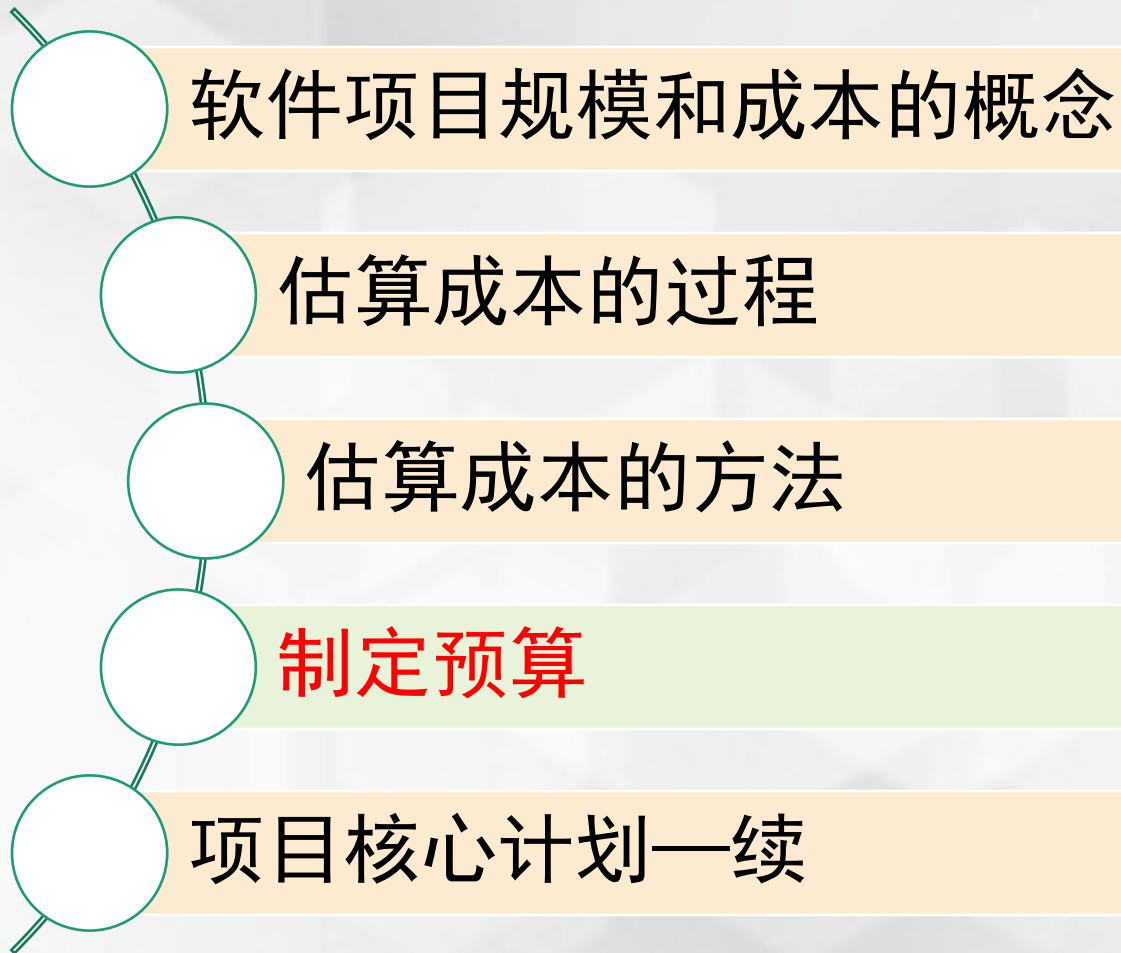


《软件工程中的真相与假象》

- **真相14:** 很少有真正的可行性分析。
- **假象6:** 要估算软件项目开发时间和开发成本，必须首先估算软件项目的代码行数。



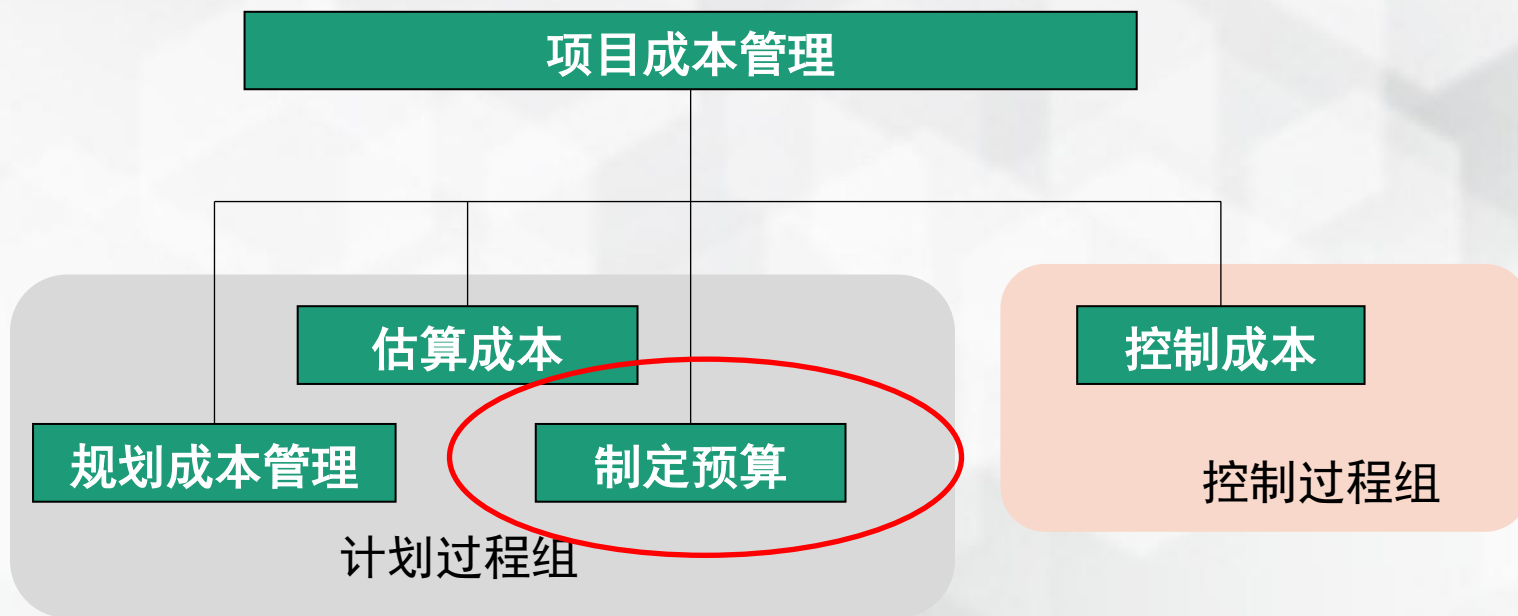
本章内容





什么是项目成本管理?

项目成本管理，就是为了在审批过的项目预算内完成项目，所进行的成本计划、估算、预算、监视和控制等一系列管理活动。





制定预算

- 制定预算是汇总所有单个活动或工作包的估算成本，建立一个**经过批准的成本基线**的过程。
- 将估算的项目总成本按照项目的进度分摊到各个工作单元中去，即将总的成本安排到各个活动中，目的是产生成本基线。

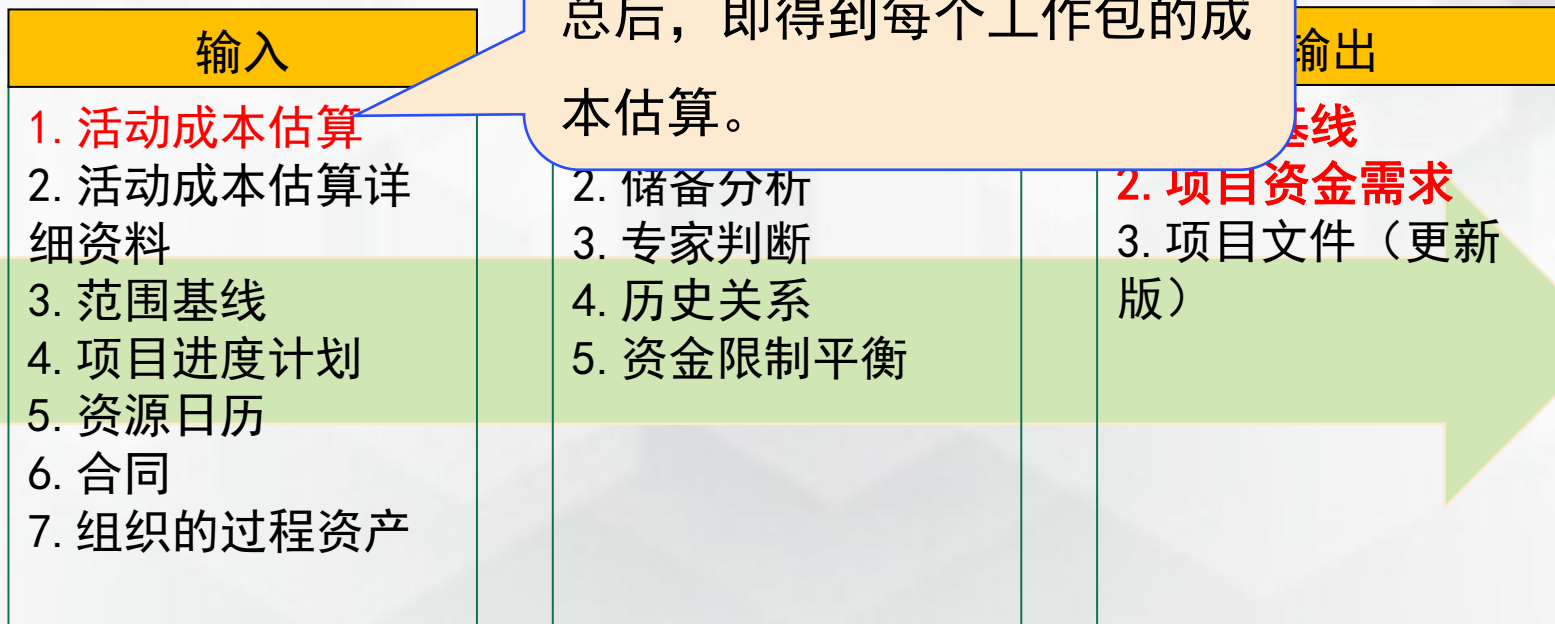
输入	工具和方法	输出
1. 活动成本估算 2. 活动成本估算详细资料 3. 范围基线 4. 项目进度计划 5. 资源日历 6. 合同 7. 组织的过程资产	1. 成本汇总 2. 储备分析 3. 专家判断 4. 历史关系 5. 资金限制平衡	1. 成本基线 2. 项目资金需求 3. 项目文件（更新版）



制定预算

- 制定预算是汇总所有单个活动或工作包的估算成本，建立一个**经过批准的成本基线**的过程。
- 将估算的项目总成本按照项目的进度分摊到各个工作单元中去，即将总的成本安排到**成本基线**。

工作包的每个活动成本估算汇总后，即得到每个工作包的成本估算。





制定预算

- 制定预算是汇总所有单个活动或工作包的估算成本，建立一个**经过批准的成本基线**的过程。
- 将估算的项目总成本按照项目的进度分摊到各个工作单元中去，即将总的成本安排到各个活动中，目的是产生成本基线。

成本估算的输出之一，重点描述成本估算的依据。

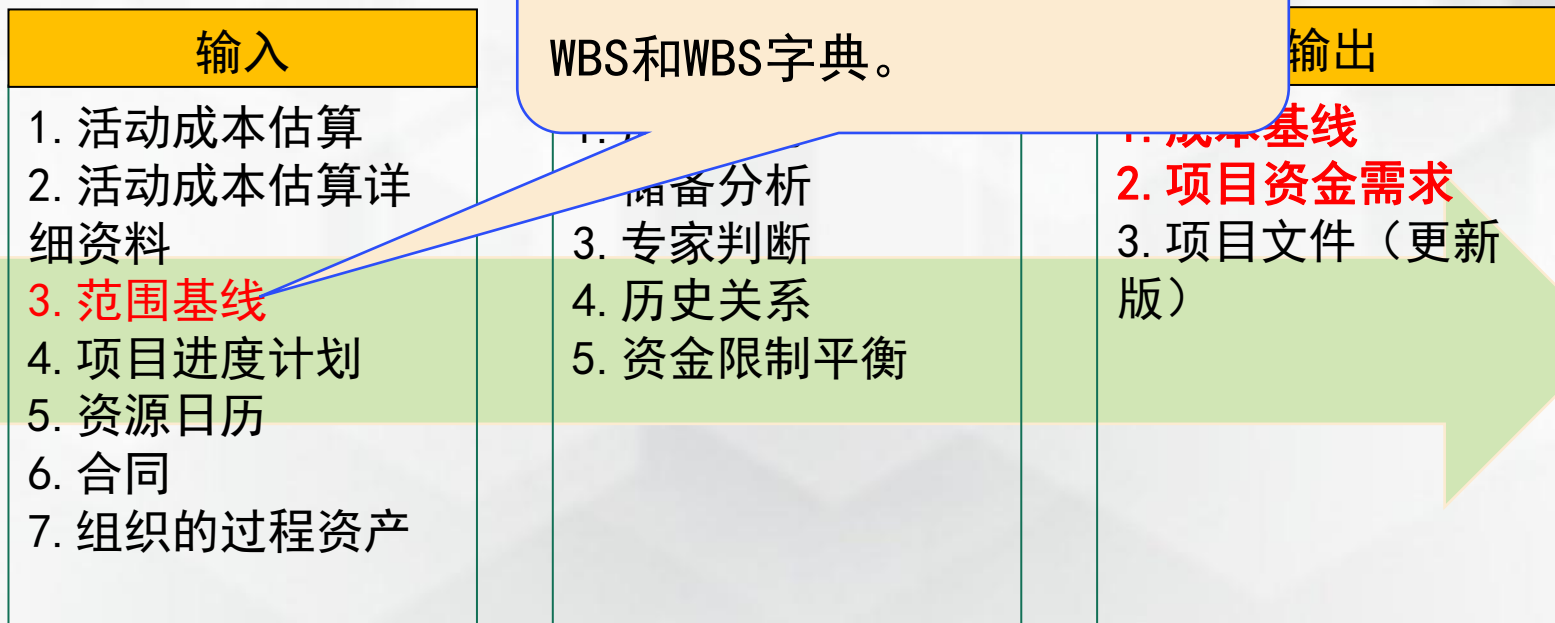
输入		输出
1. 活动成本估算 2. 活动成本估算详细资料 3. 范围基线 4. 项目进度计划 5. 资源日历 6. 合同 7. 组织的过程资产	2. 储备分析 3. 专家判断 4. 历史关系 5. 资金限制平衡	1. 成本基线 2. 项目资金需求 3. 项目文件（更新版）



制定预算

- 制定预算是汇总所有单个活动或工作包的估算成本，建立一个**经过批准的成本基线**的过程。
- 将估算的项目总成本按照项目的进度分摊到各个工作单元中去，即将总的成本安排到各个活动上，以形成**成本基线**。

包括批准的需求规格说明书，
WBS和WBS字典。



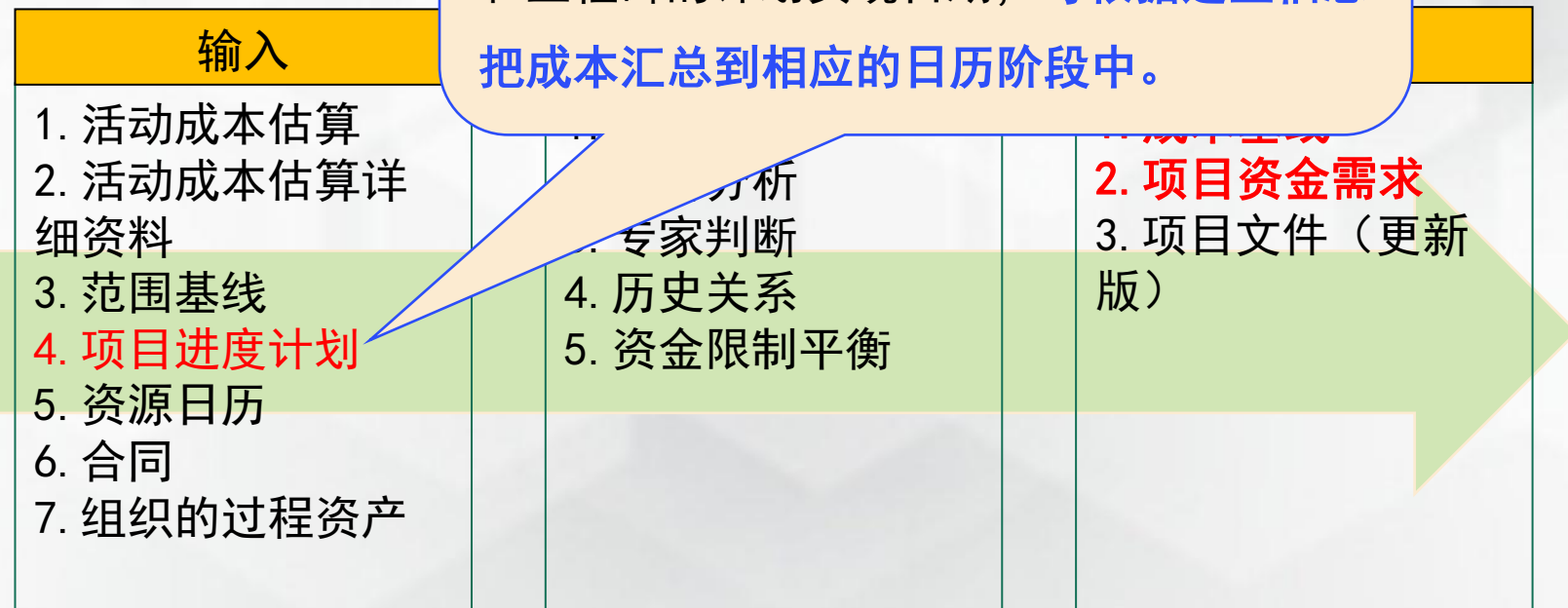


制定预算

- 制定预算是汇总所有单个活动或工作包的估算成本，建立一个**经过批准的成本基线**的过程。

- 将估算的项目总成本分配到工作包或活动的成本基线中，即将总的成本安

项目进度管理计划过程的输出。包括项目活动和工作包的计划开始时间、计划完成时间和里程碑的计划实现日期，**可根据这些信息把成本汇总到相应的日历阶段中。**





制定预算

- 制定预算是汇总所有单个活动或工作包的估算成本，建立一个**经过批准的成本基线**的过程。
- 将估算的项目总成本按照项目的进度分摊到各个工作单元中去，即将总的成本安排

从资源日历中了解项目资源的种类和使用时间。可根据这些信息确定项目周期各个阶段的资源成本。

线。

输入
1. 活动成本估算
2. 活动成本估算详细资料
3. 范围基线
4. 项目进度计划
5. 资源日历
6. 合同
7. 组织的过程资产

2. 项目资源需求
3. 项目文件（更新版）
4. 历史关系
5. 资金限制平衡

2. 项目资金需求
3. 项目文件（更新版）



制定预算

- 制定预算是汇总所有单个活动或工作包的估算成本，建立一个**经过批准的成本基线**的过程。
- 将估算的项目总成本按照项目的进度分摊到各个工作单元中去，即将总的成本安排到各个活动中，目的是建立成本基线。

在制定预算时，需要考虑已经采购产品的成本和相关合同信息。

输入
1. 活动成本估算
2. 活动成本估算详细资料
3. 范围基线
4. 项目进度计划
5. 资源日历
6. 合同
7. 组织的过程资产

2. 专家判断
4. 历史关系
5. 资金限制平衡

输出
2. 项目资金需求
3. 项目文件（更新版）



制定预算

- 制定预算是汇总所有单个活动或工作包的估算成本，建立一个**经过批准的成本基线**的过程。
- 将估算的项目总成本按照项目的进度分摊到各个工作单元中去，即将总的成本安排到项目的时间表上，形成**成本基线**。

- 现有的正式的或非正式的，与成本预算相关的政策、程序和指南
- 成本预算工具
- 报告方法

输入

1. 活动成本估算
2. 活动成本估算详细资料
3. 范围基线
4. 项目进度计划
5. 资源日历
6. 合同
7. 组织的过程资产

2. 项目资金需求
 3. 项目文件（更新版）
- 2. 项目资金需求
 - 3. 项目文件（更新版）

2. 项目资金需求
3. 项目文件（更新版）



制定预算

- 制定预算是汇总所有单元的成本，并建立一个**经过批准的成本基线**。
- 将估算的项目总成本分配到工作包中，并将总的成本安排到项目预算中去，得到**项目成本基线**。

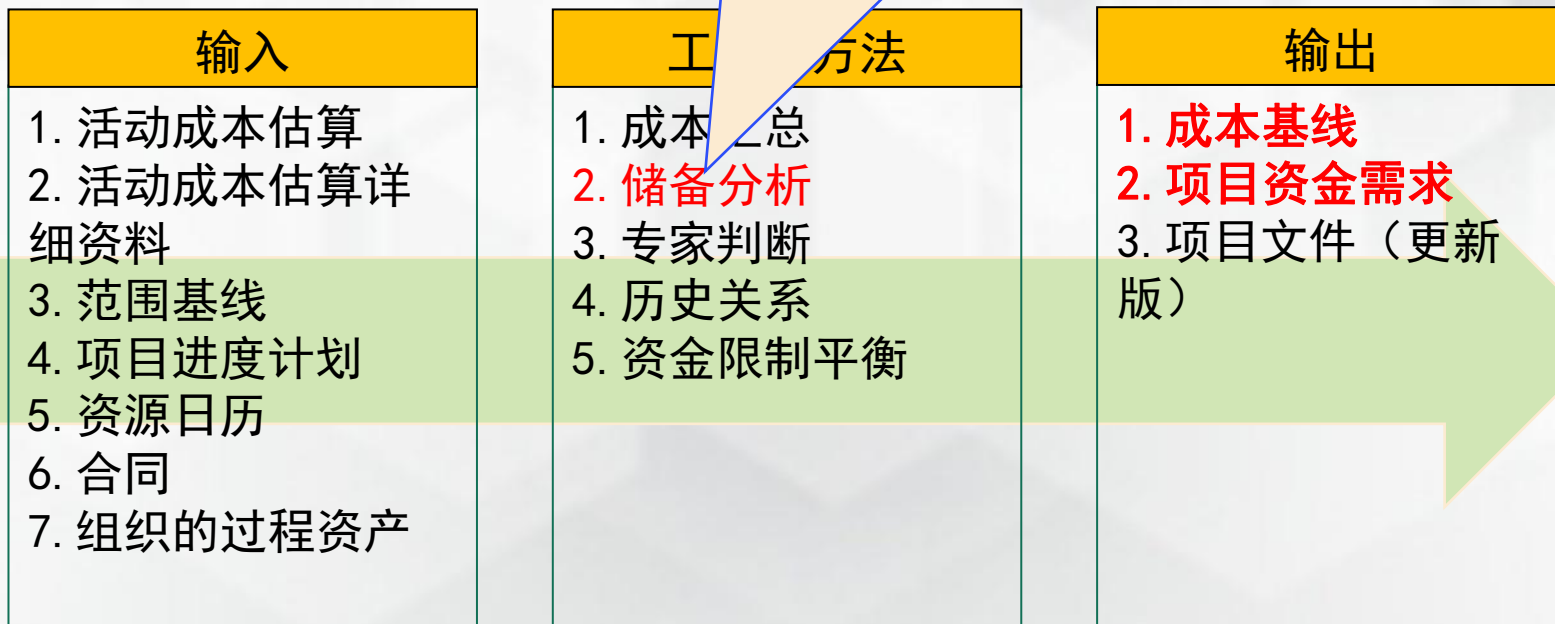
- 首先，以WBS中的工作包为单位对活动成本估算进行汇总。
- 然后由工作包汇总到WBS的更高层次，并最终得到项目的总成本。

输入	工具和方法	输出
1. 活动成本估算 2. 活动成本估算详细资料 3. 范围基线 4. 项目进度计划 5. 资源日历 6. 合同 7. 组织的过程资产	1. 成本汇总 2. 储备分析 3. 专家判断 4. 历史关系 5. 资金限制平衡	1. 成本基线 2. 项目资金需求 3. 项目文件（更新版）



制定预算

- 制定预算是汇总所有单个活动或工作包的估算成本，建立一个**经过批准的成本基线**的过程。
- 将估算的项目总成本按**通过预算储备分析，可以及时掌握所需单元中去的应急储备和管理储备。**，即将总的成本安排到**基线。**

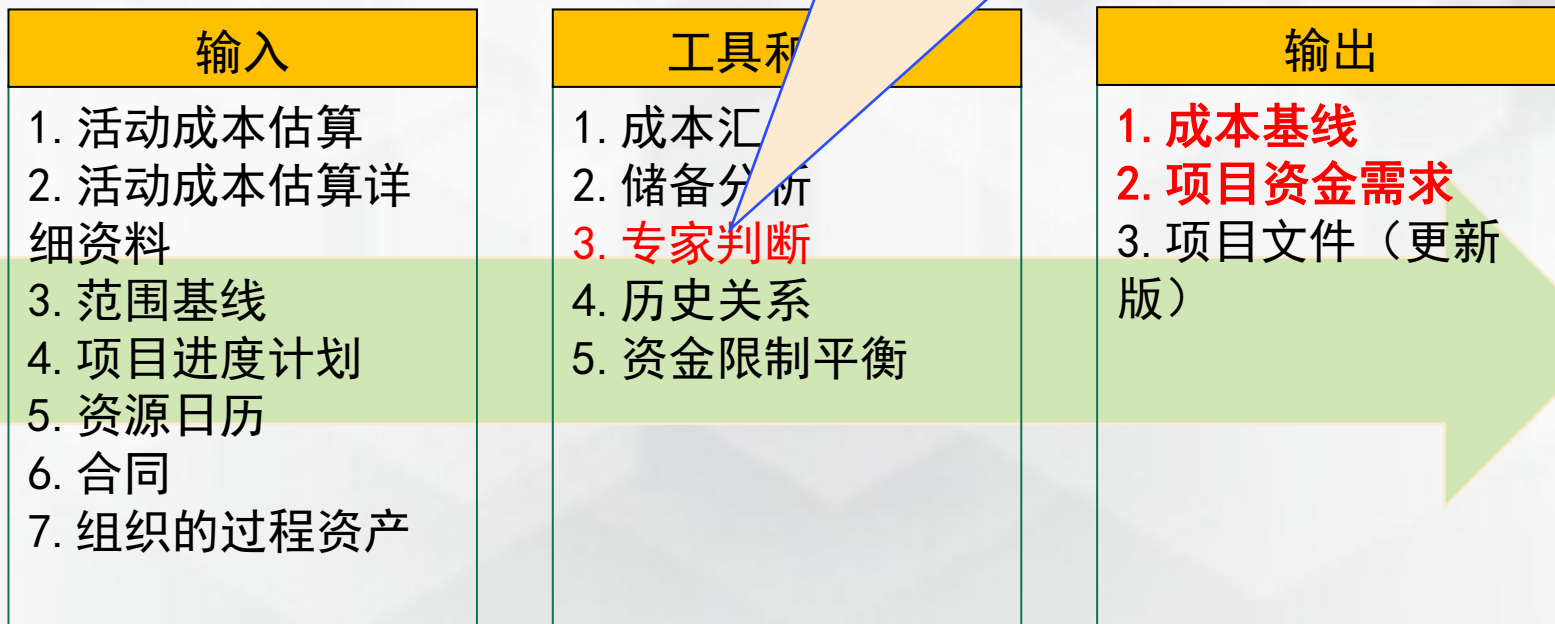




制定预算

- 制定预算是汇总所有单个活动或工作包的估算成本，建立一个**经过批准的成本基线**的过程
- 将估算的项目总成本按照项目分解，即将总的成本安排到各个工作包中去

在制定预算的过程中，应根据项目的工作需要，基于应用领域、知识领域、学科、行业等专业知识做出专家判断。



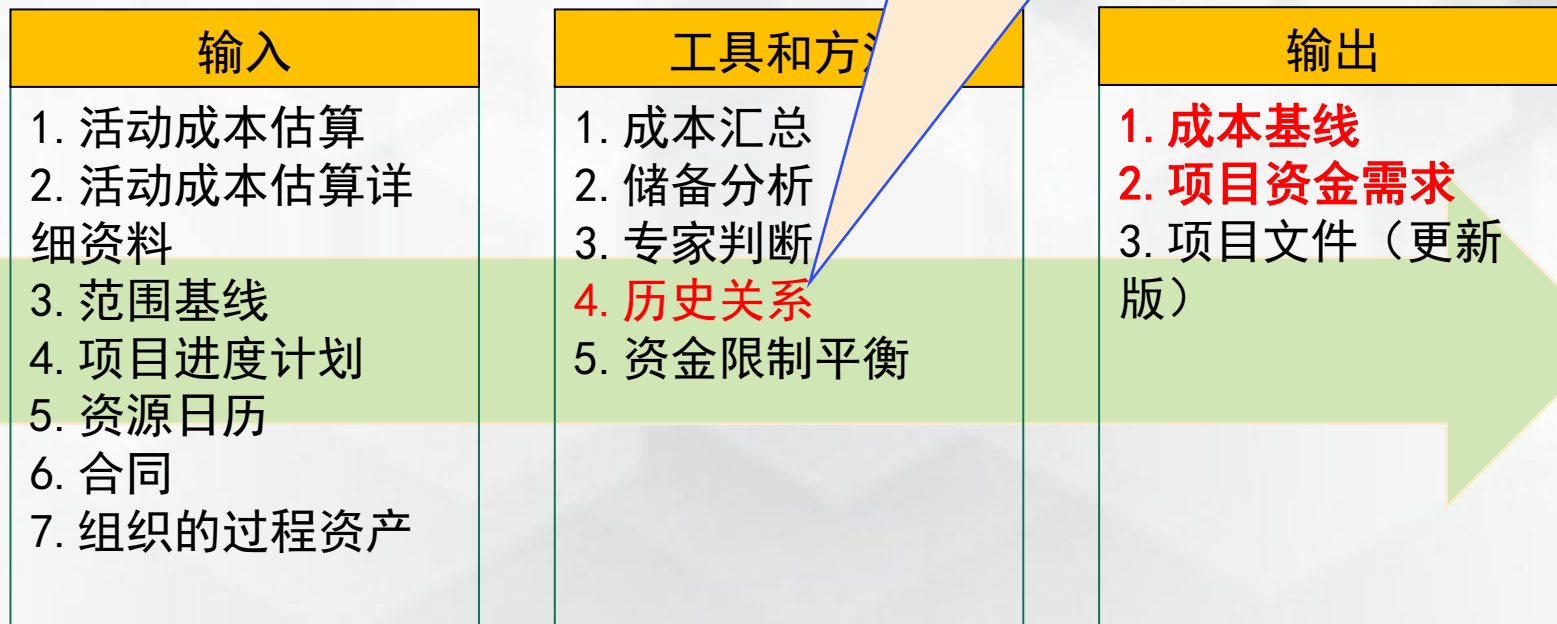


制定预算

- 制定预算是汇总所有单个活动或工作包的估算成本，建立一个**经过批准的成本基线**的过程。
- 将估算的项目总成本按照项目，即将总的成本安排到各个活

可使用历史数据来建立成本
预算模型。

单元中去
基线。

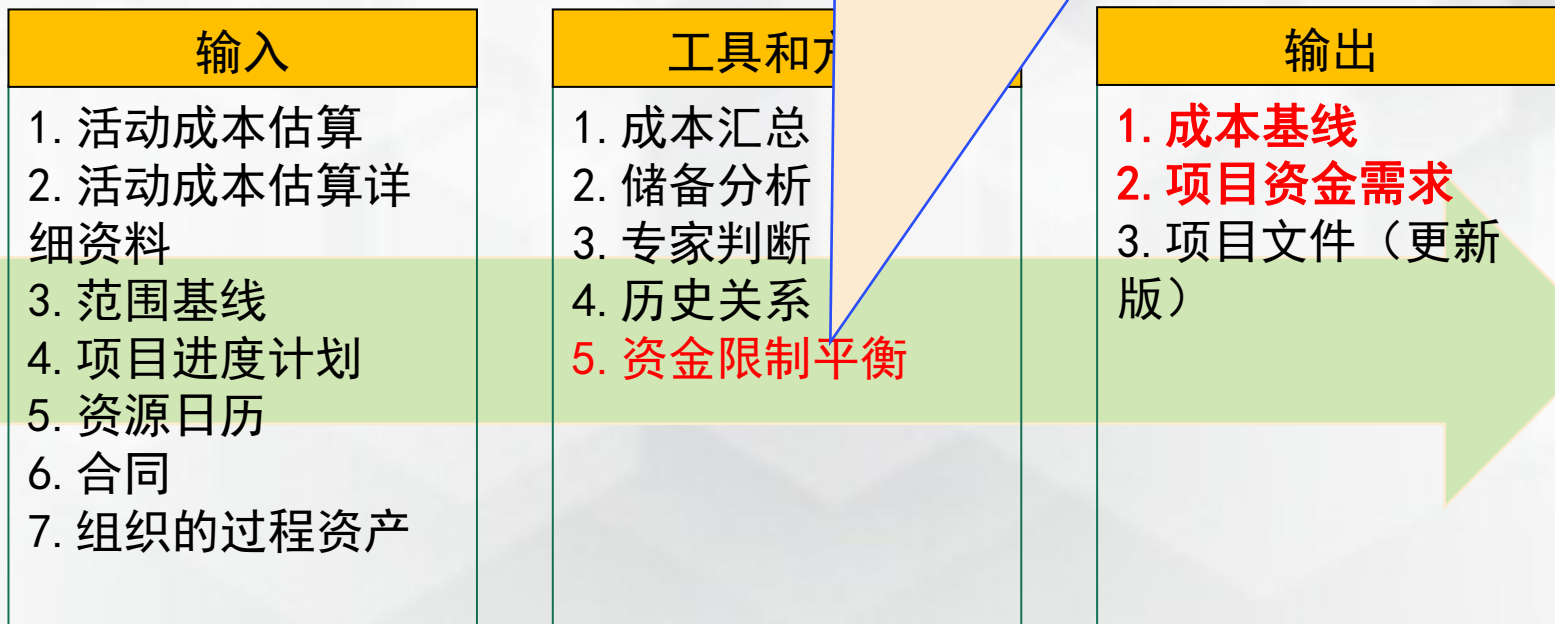




制定预算

- 制定预算是汇总所有单个活动成本估算，**经过批准的成本基线**的过程。
- 将估算的项目总成本按照资源可用性进行分配，即将总的成本安排到各工作包中。

- 应该根据对项目资金的限制，来平衡资金的支出。
- 如果发现资金限制与计划支出有差异，则可能需要调整工作的进度计划，以平衡资金支出水平。





制定预算

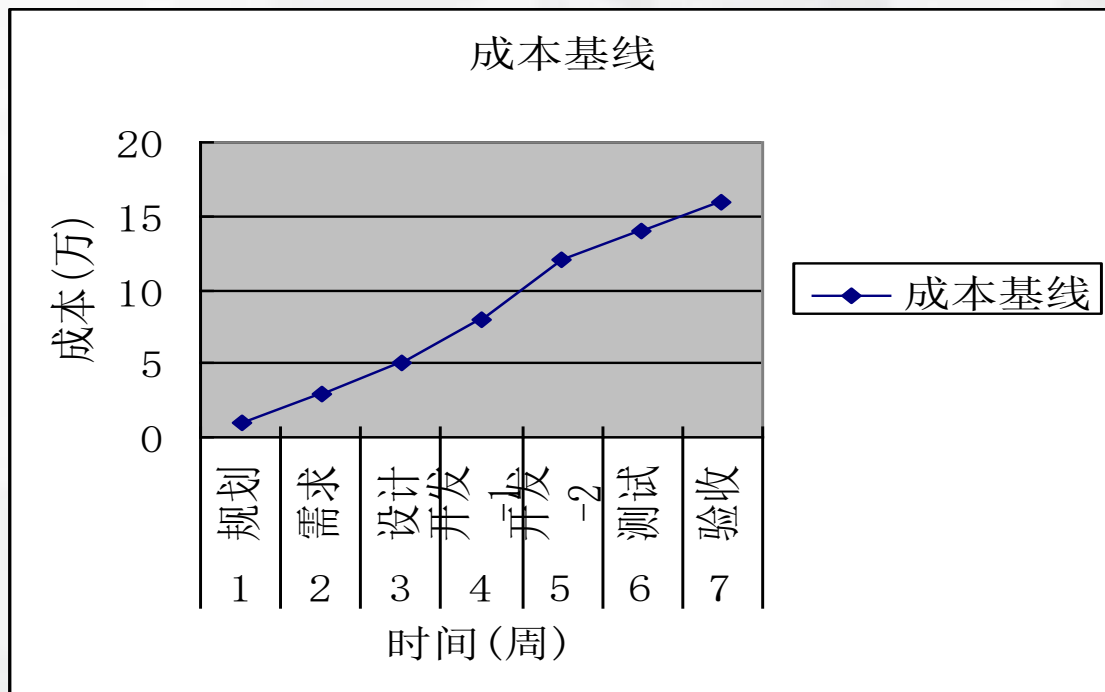
- 制定预算是汇总所有单个活动或工作包的估算成本，建立一个**经过批准的成本基线**的过程。
- 将估算的项目总成本按照项目的进度分摊到各个工作单元中去，即将总的成本安排到各个活动中，目的是产生成本基线。

输入	工具和方法	输出
1. 活动成本估算 2. 活动成本估算详细资料 3. 范围基线 4. 项目进度计划 5. 资源日历 6. 合同 7. 组织的过程资产	1. 成本汇总 2. 储备分析 3. 专家判断 4. 历史关系 5. 资金限制平衡	1. 成本基线 2. 项目资金需求 3. 项目文件（更新版）



1. 成本基线

- 是经过批准的且按时间段分配资金的预算。
- 它被用来测量、监督和控制项目的总体成本绩效。
- 它是每个时间段预算的总和，通常用S曲线表示。





制定预算

- 制定预算是汇总所有单个活动或工作包的估算成本，建立一个**经过批准的成本基线**的过程。
- 将估算的项目总成本按照项目的进度分摊到各个工作单元中去，即将总的成本安排到各个活动中，目的是产生成本基线。

输入	工具和方法	输出
1. 活动成本估算 2. 活动成本估算详细资料 3. 范围基线 4. 项目进度计划 5. 资源日历 6. 合同 7. 组织的过程资产	1. 成本汇总 2. 储备分析 3. 专家判断 4. 历史关系 5. 资金限制平衡	1. 成本基线 2. 项目资金需求 3. 项目文件（更新版）



2. 项目资金需求

- 根据成本基线，确定总资金需求和阶段性（如季度或者年度）资金需求。
- 项目的资金投入通常以增量而非连续的方式进行，故呈现阶梯状。
- 如有管理储备，则总资金需求等于成本基线加上管理储备。



提问

制定成本预算的主要目的是（ A ）。

- A、产生成本基线
- B、编写报告书
- C、指导设计过程
- D、方便进度管理



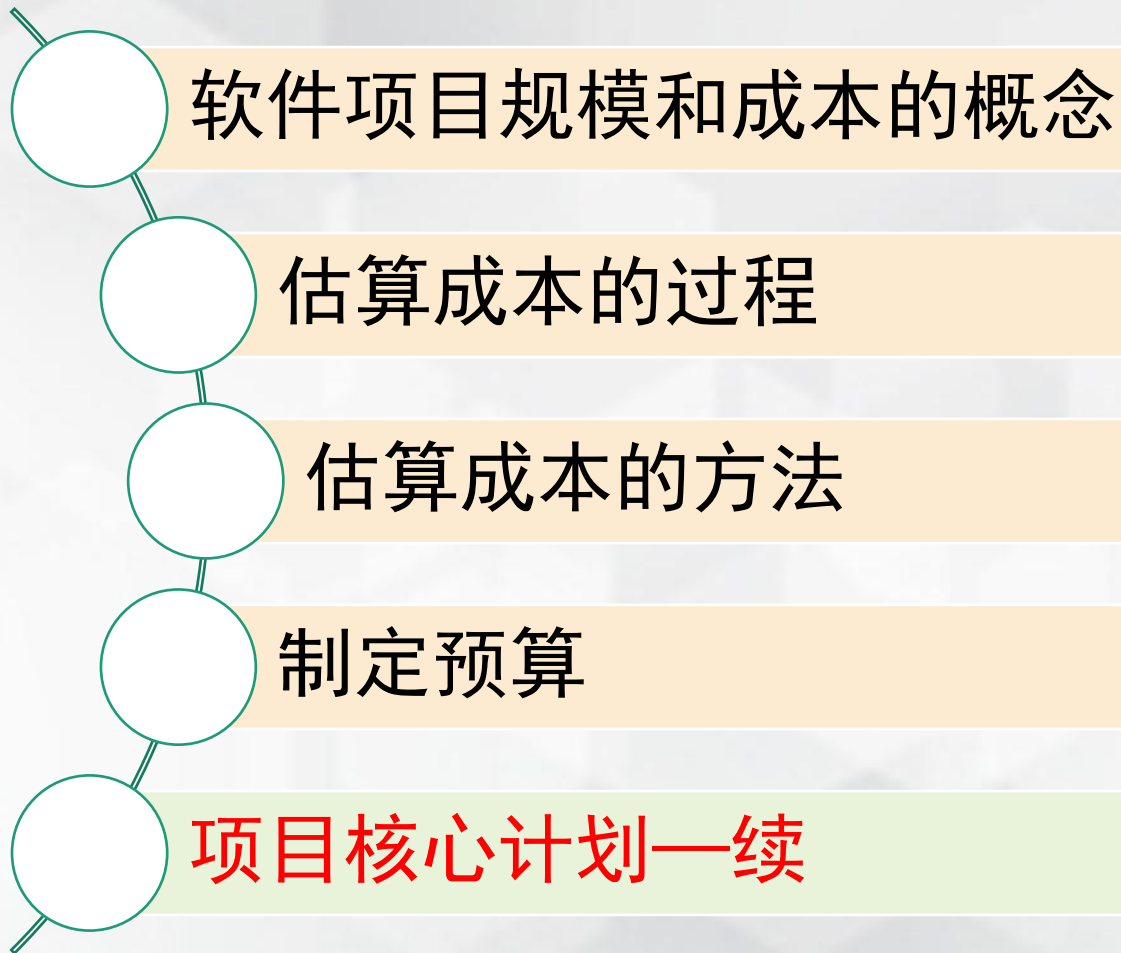
提问

在项目成本管理过程中，每个时间段中的各个工作单元的成本是（ B ）。

- A、估算
- B、预算
- C、直接成本
- D、间接成本



本章内容





编制项目核心计划的步骤

第7章已做讲解！

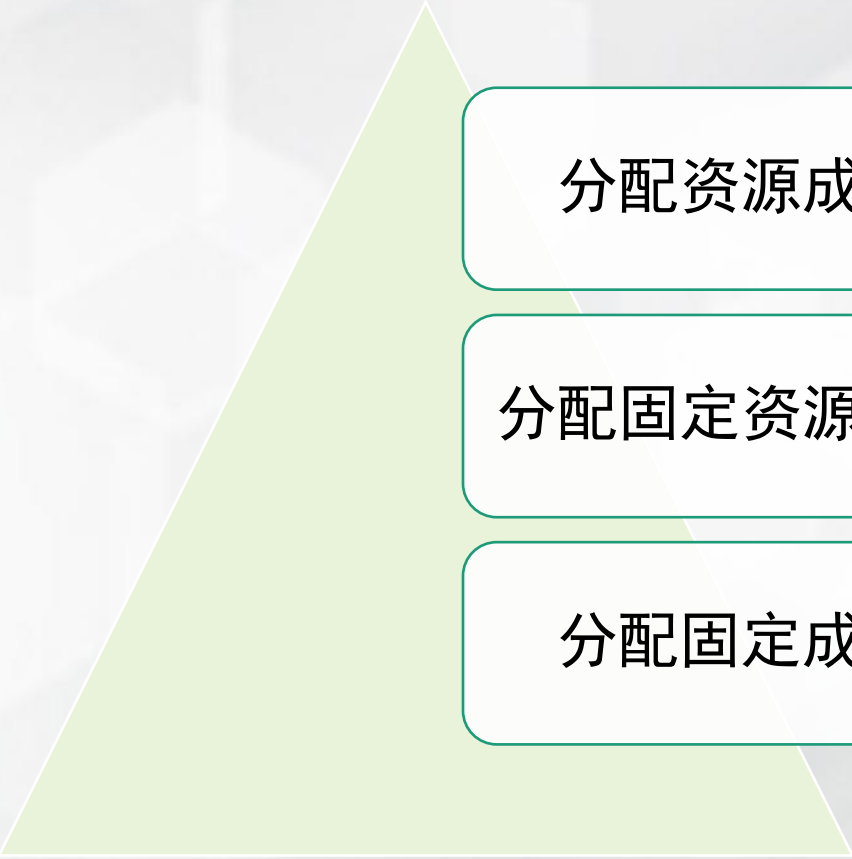




制定成本预算

结合进度重新调整项目成本预算。

分配项目成本预算包括三种情况：



分配资源成本

分配固定资源成本

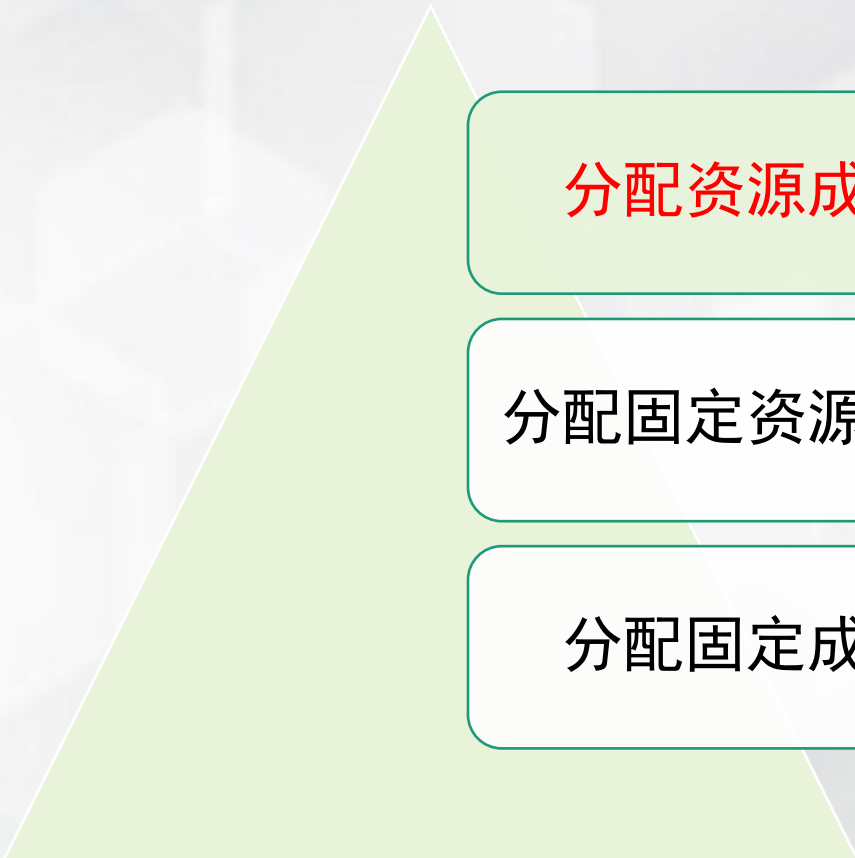
分配固定成本



制定成本预算

结合进度重新调整项目成本预算。

分配项目成本预算包括三种情况：



分配资源成本

分配固定资源成本

分配固定成本



分配资源成本

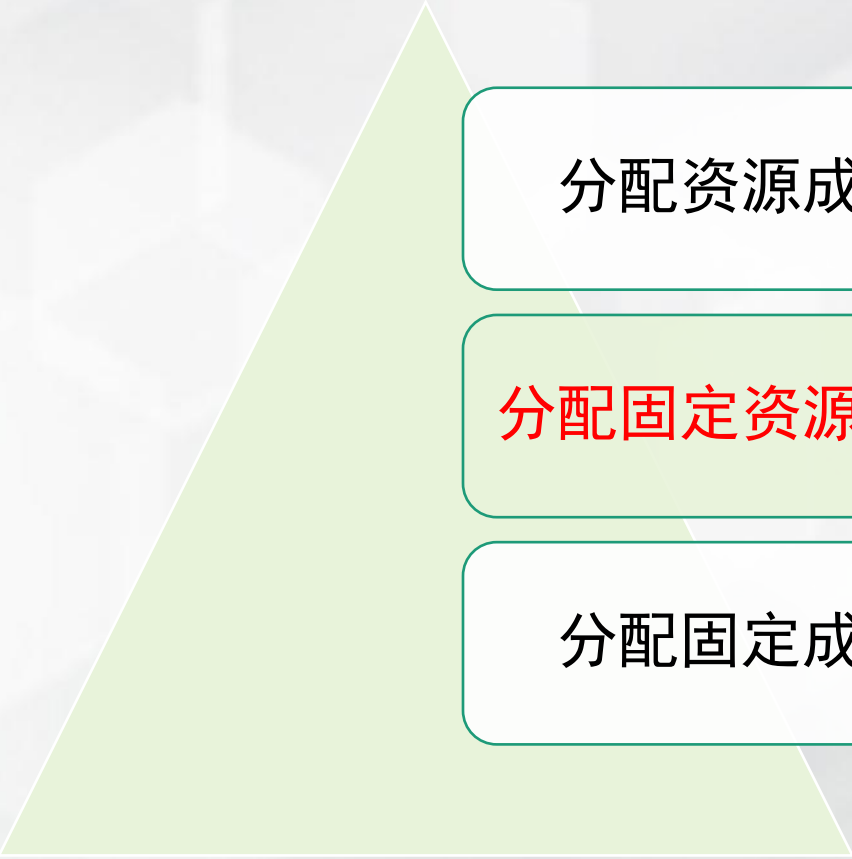
- 这是最常用的一种方式，即根据每个活动的资源分配情况来计算这个活动的成本预算。
- 资源成本与资源的基本费率紧密相连的，所以要设置资源费率。例如：
 - 标准费率（如项目开发人员的标准费率为200元/小时）
 - 加班费率
 - 每次使用费率
 -



制定成本预算

结合进度重新调整项目成本预算。

分配项目成本预算包括三种情况：



分配资源成本

分配固定资源成本

分配固定成本



分配固定资源成本

当一个项目的资源需要固定数量的资金时，可以向活动分配固定资源成本。

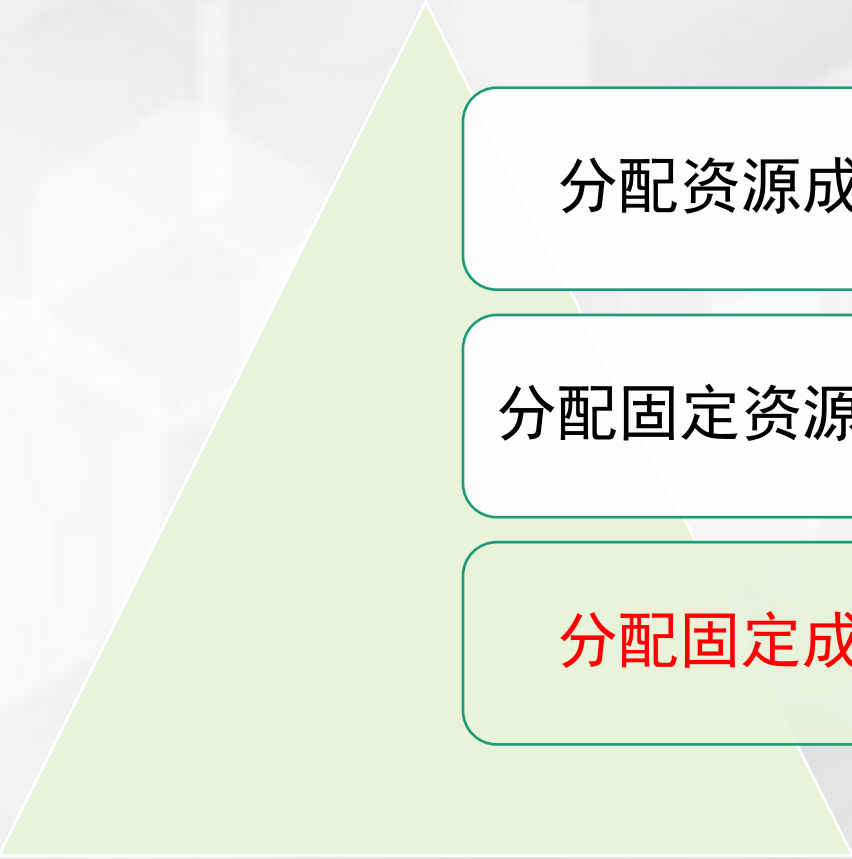
例如：需要的硬件设备



制定成本预算

结合进度重新调整项目成本预算。

分配项目成本预算包括三种情况：



分配资源成本

分配固定资源成本

分配固定成本



分配固定成本

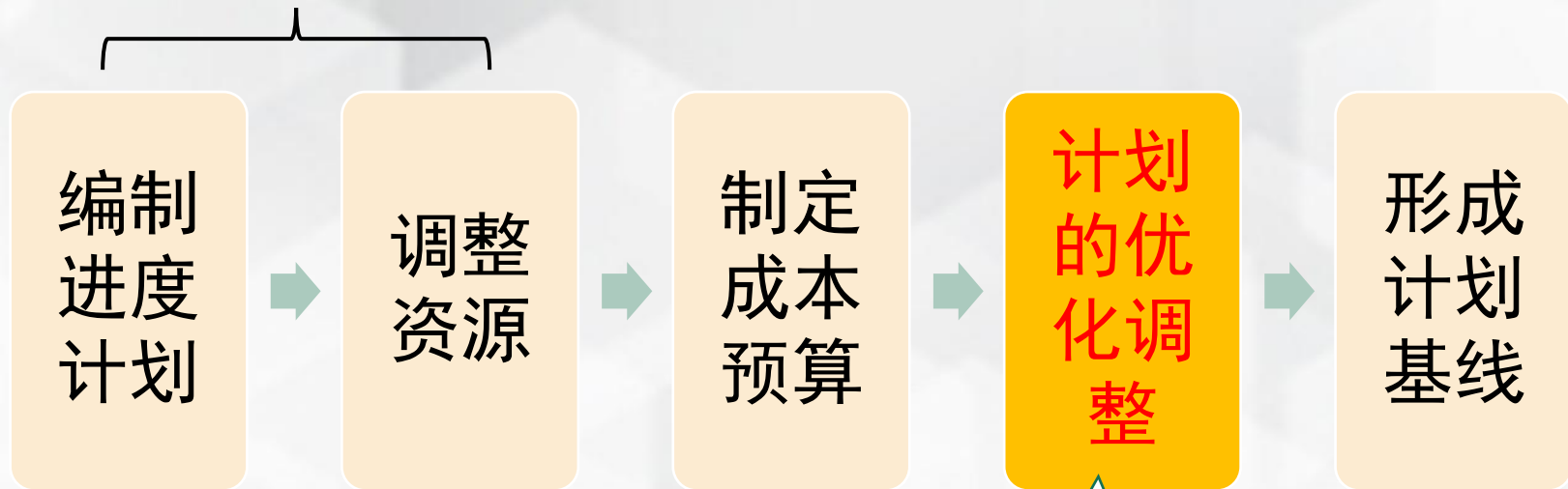
有些活动是固定成本的类型的活动，也就是说，某项活动的成本不变，不管活动的工期有多长，或不管活动使用了哪些资源。在这种情况下，可以向活动直接分配成本。

例如：培训任务



编制项目核心计划的步骤

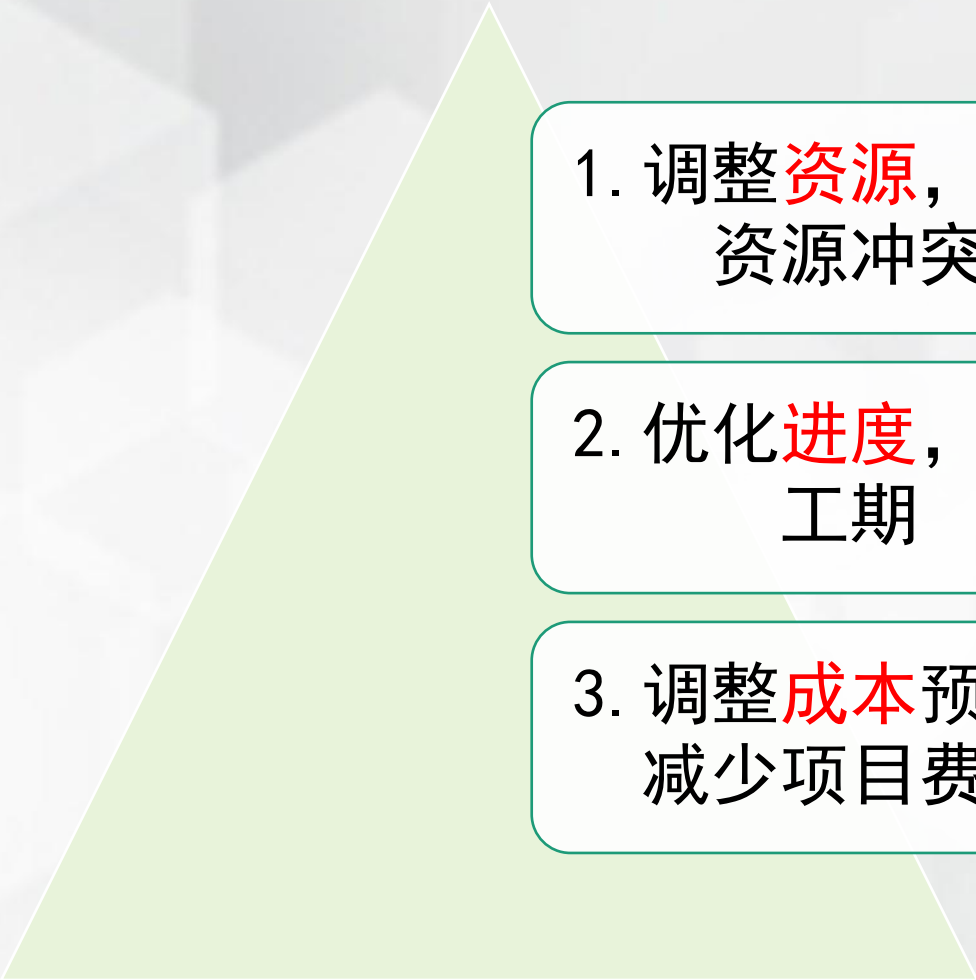
第7章已做讲解！



需要从资源、进度、成本
三个方面去综合考虑！



计划的优化调整

- 
1. 调整**资源**，解决资源冲突
 2. 优化**进度**，缩短工期
 3. 调整**成本**预算，减少项目费用



计划的优化调整

1. 调整资源，解决
资源冲突

2. 优化进度，缩短
工期

3. 调整成本预算，
减少项目费用



1.调整资源，解决资源冲突

资源冲突(过度分配)主要有两种表现：

- 分配给一个资源的工时总量大于它的最大可用工时量。
- 同一种资源被分配给时间上重叠的几个活动或项目中。



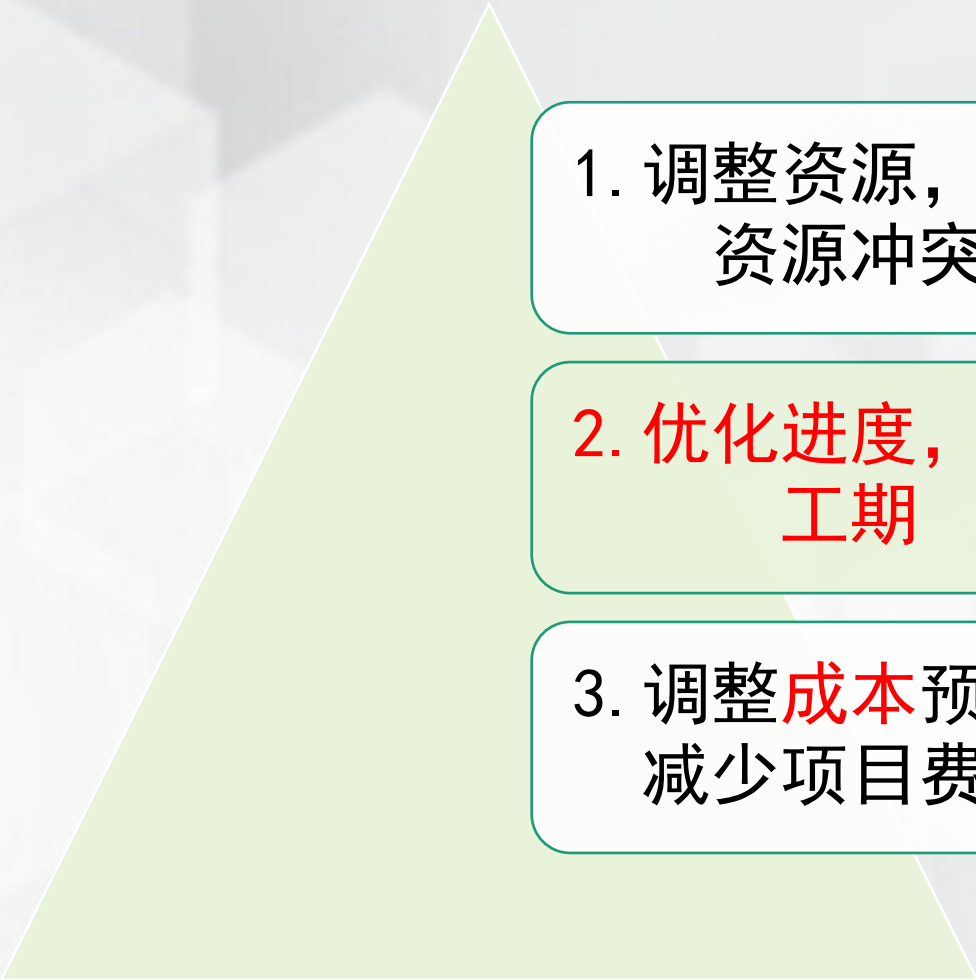
1.调整资源，解决资源冲突

解决资源冲突的措施：

- 资源调配；
- 推迟资源开始工作时间；
- 替换资源；
- 设置资源加班时间；
- 调整资源日历；
- 只使用资源的一部分工作时间。



计划的优化调整



1. 调整资源，解决
资源冲突

2. 优化进度，缩短
工期

3. 调整**成本**预算，
减少项目费用



2.优化进度，缩短工期

- 检查项目中各活动的执行时间是否合理，有无冲突现象；
- 尽可能缩短项目工期。



2. 优化进度， 缩短工期

常用办法：

1. 分解关键活动；
2. 给活动增加资源；
3. 缩短关键活动的工期；
4. 重叠或超前关键活动；
5. 设置日历增加工作时间；
6. 通过分配加班工时来缩短关键活动。



2. 优化进度， 缩短工期

常用办法：

1. 分解关键活动；
2. 给活动增加资源；
3. 缩短关键活动的工期；
4. 重叠或超前关键活动；
5. 设置日历增加工作时间；
6. 通过分配加班工时来缩短关键活动。

通过“分解关键活动”
可以缩短活动工期，但
有时候，受资源的限制
，有些活动是不能同步
进行的，所以这时活动
分解也无助于缩短项目
工期。



2. 优化进度， 缩短工期

常用办法：

1. 分解关键活动；
 2. 给活动增加资源；
 3. 缩短关键活动的工期
 4. 重叠或超前关键活动
 5. 设置日历增加工作时
 6. 通过分配加班工时来
- 增加的资源数量不能大于资源的最大可用量。
 - 增加资源必须是主导项目工期的关键路径上。
 - 关键活动的缩短可能会变成非关键任务，因此，此时增加过多的资源也无法达到继续缩短总工期的目的。



2. 优化进度， 缩短工期

常用办法：

1. 分解关键活动；
2. 给活动增加资源；
3. 缩短关键活动的工期；
4. 重叠或超前关键活动；
5. 设置日历增加工作时间；
6. 通过分配加班工时来缩短关键活动。

在活动已分配了资源的情况下，缩短活动工期意味着增加资源的工作量，可能导致资源的过度分配。



2. 优化进度， 缩短工期

常用办法：

1. 分解关键活动；
 2. 给活动增加资源；
 3. 缩短关键活动的工期；
 4. 重叠或超前关键活动；
 5. 设置日历增加工作时间；
 6. 通过分配加班工时来缩短关键活动。
- 改变活动的链接关系；
 - 在链接活动之间增加超前。



2. 优化进度， 缩短工期

常用办法：

1. 分解关键活动；
2. 给活动增加资源；
3. 缩短关键活动的工期；
4. 重叠或超前关键活动；
5. 设置日历增加工作时间；
6. 通过分配加班工时来缩短关键活动。

可以通过改变资源的日历来调整工期，比如将资源原来的休息时间改变成工作时间为实现。这样通过增加资源的工作时间来缩短任务的工期。



2. 优化进度， 缩短工期

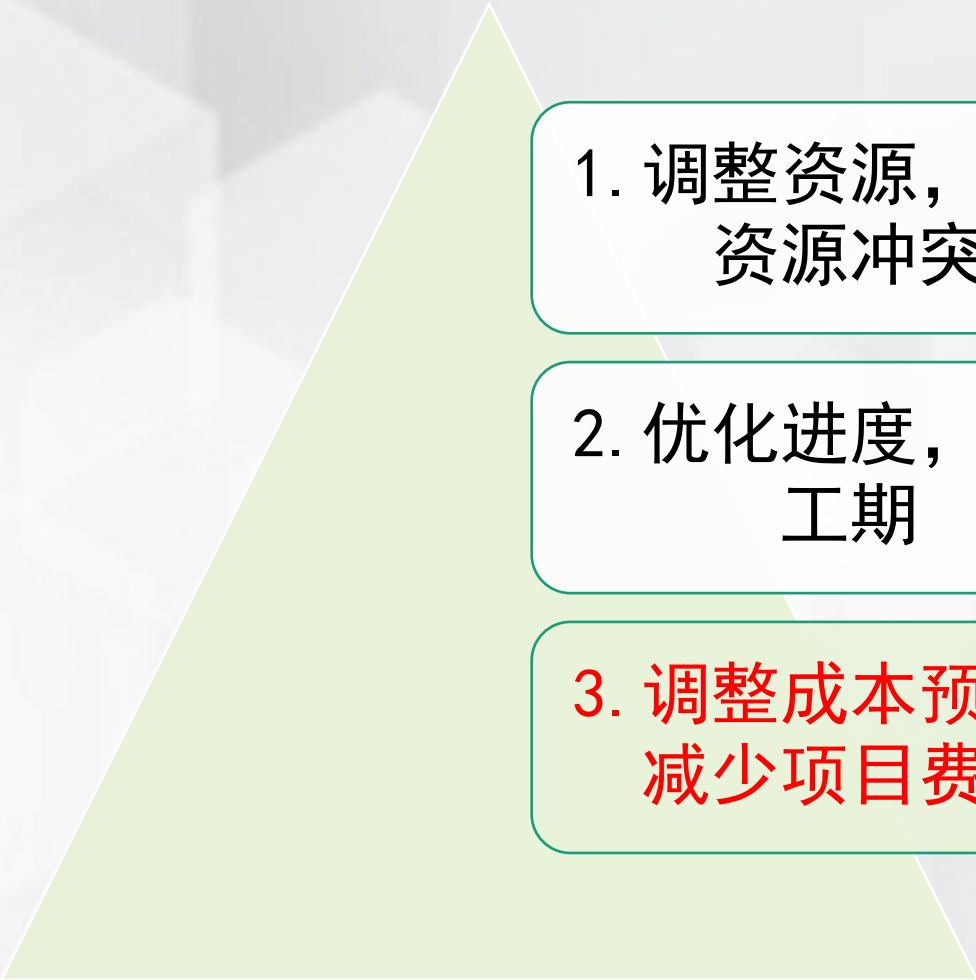
常用办法：

1. 分解关键活动；
2. 给活动增加资源；
3. 缩短关键活动的工期；
4. 重叠或超前关键活动；
5. 设置日历增加工作时间；
6. 通过分配加班工时来缩短关键活动。

需要在关键活动上为资源设置加班时间，以缩短活动工期。



计划的优化调整



1. 调整资源，解决
资源冲突

2. 优化进度，缩短
工期

3. 调整成本预算，
减少项目费用



3.调整成本预算，减少项目费用

降低预算成本的常用方法：

1. 降低资源的费率；
2. 减少活动的工时；
3. 减少加班；
4. 替换资源；
5. 减少活动的固定成本；
6. 删除活动。



3.调整成本预算，减少项目费用

降低预算成本的常用方法：

1. 降低资源的费率；
2. 减少活动的工时；
3. 减少加班；
4. 替换资源；
5. 减少活动的固定成本；
6. 删除活动。

降低资源的费率往往会打击工作人员的积极性，但可以通过降低其他资源的费率来实现，比如降低能源消耗、设备费用等。



3.调整成本预算，减少项目费用

降低预算成本的常用方法：

1. 降低资源的费率；
2. 减少活动的工时；
3. 减少加班；
4. 替换资源；
5. 减少活动的固定成本；
6. 删除活动。

适当的减少工时，可以降低活动的费用。但减少工时同时也影响项目的工期。



3.调整成本预算，减少项目费用

降低预算成本的常用方法：

1. 降低资源的费率；
2. 减少活动的工时；
3. 减少加班；
4. 替换资源；
5. 减少活动的固定成本；
6. 删除活动。

加班需要支付加班费，加班费率通常要高于资源费率，所以减少加班可以有效地减少任务成本。



3.调整成本预算，减少项目费用

降低预算成本的常用方法：

1. 降低资源的费率；
2. 减少活动的工时；
3. 减少加班；
4. 替换资源；
5. 减少活动的固定成本；
6. 删除活动。

用廉价的资源替换比较高价的资源，但有一个前提，那就是替换的资源同样能胜任这项任务。



3.调整成本预算，减少项目费用

降低预算成本的常用方法：

1. 降低资源的费率；
2. 减少活动的工时；
3. 减少加班；
4. 替换资源；
5. 减少活动的固定成本；
6. 删除活动。

固定成本就是活动本身所需要的成本。



3.调整成本预算，减少项目费用

降低预算成本的常用方法：

1. 降低资源的费率；
2. 减少活动的工时；
3. 减少加班；
4. 替换资源；
5. 减少活动的固定成本；
6. 删除活动。

确认删除该活动对项目没有影响或影响在可控制范围内才可采用。



提问

下面的哪一项不是解决资源冲突的常用办法

(C) 。

- A、替换资源
- B、推迟资源开始工作时间
- C、降低资源的费率
- D、调整资源日历



提问

下面的哪一项不是优化进度，缩短工期的常用方法（ B ）。

- A、分解关键活动
- B、推迟资源开始工作时间
- C、设置日历增加工作时间
- D、缩短关键活动的工期



提问

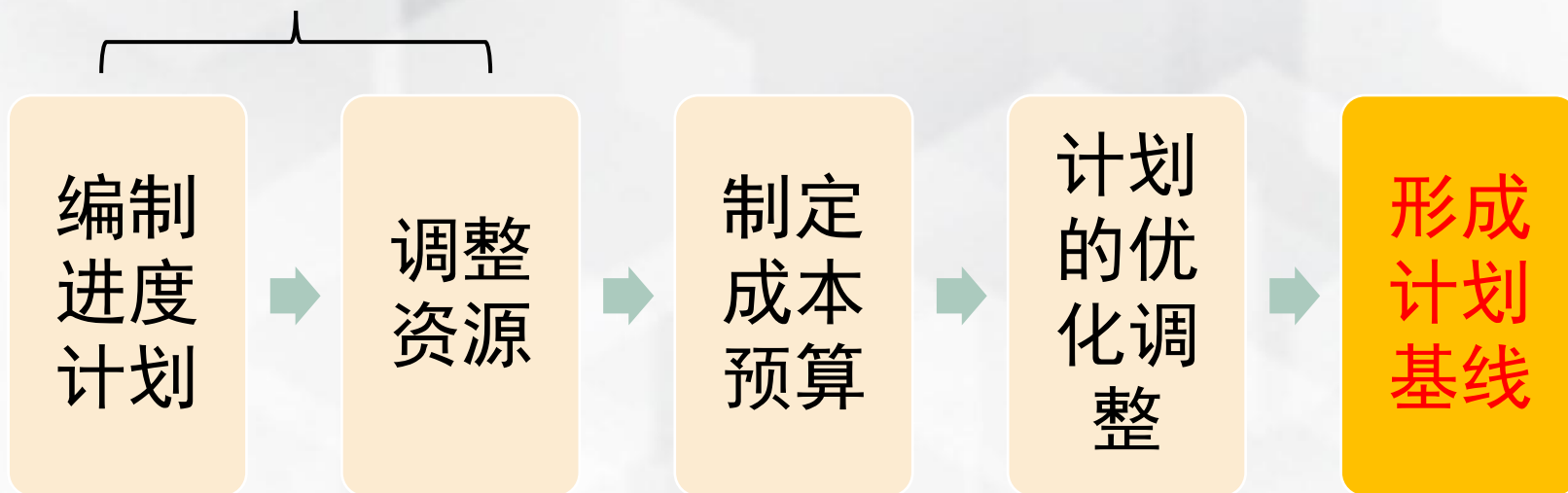
下面的哪一项不是降低预算成本的常用方法
(A) 。

- A、给活动增加资源
- B、减少活动的工时
- C、减少加班
- D、降低资源的费率



编制项目核心计划的步骤

第7章已做讲解！





形成计划基线

- 项目计划是为了实现项目目标，根据已获得的项目信息，对未来项目过程所做出的假设。相关信息越可靠、越真实，相关环境越稳定，这种前瞻性的假设就越可靠，与将来的实际差距也就越小。
- 在制定项目计划时，经常会在总体的里程碑计划框架下，分阶段制定详细计划。前一阶段结束了，再根据前一阶段的结果制定下一阶段的具体计划。（滚动式计划）



形成计划基线

- 整个项目的**计划基线**是由若干指标组成的，每个指标本身也是基线。例如：项目范围基线、产品定义基线、时间进度基线、资源基线、成本基线等，所有这些基线共同组成了整个项目的基线。
- 针对同一个项目计划，可能存在多个基线，不同管理层可能有不同的基线，不同时期可能有不同的基线。



形成计划基线

- 最后的项目计划一定要与重要的项目干系人进行确认，**尤其是客户的确认**。确认后的项目计划，要作为以后项目执行的基准。如有变更，就要走变更流程。计划如果没有与客户确认，就可能会出现客户随意变更的情况。



核心三计划总结

